

NEW-AGE GLOBAL UNIVERSITY FOR LIBERAL EDUCATION

Semester: II
Operating System
Category: Core
Course Code: CS1103
(Theory and Lab)

Unit – I

- Introduction to Operating Systems: Operating Systems –Definition, Types,Functions ,Abstract view of OS,System Structures,System Calls,Virtual Machines.

Introduction to Operating Systems

- An operating system is **software** that **manages a computer's hardware**. It also provides a **basis for application programs and acts as an intermediary between the computer user and the computer hardware**.
- The operating system provides the **means for proper use of these resources** in the operation of the computer system.
- An **operating system** is similar to a **government**. Like a government, it performs no useful function by itself. It simply provides an environment within which other programs can do useful work.

Few Famous Operating Systems

Few laptop / desktop OS

Few Mobile OS

	Logo	Company	OS		Logo	Company	OS
1.		Microsoft	Windows			Google	Android
2.		Apple Inc.	 MacOS			Apple Inc.	iOS
3.		Open Source (Linux based)	ubuntu	The most popular RTOS			
4.		Open Source (ubuntu based)	LINUX LITE simple - fast - free			WIND™ AN INTEL COMPANY	VxWorks
5.		Open Source (Linux based)	fedora	RTOS: Real-time OS			
Note: Android is built on top of modified version of Linux kernel				RTOS is meant for embedded devices which have real-time constraints			
				Note: Intel acquired Wind River in 2009			
				Ref: iOS, tvOS, WatchOS			
				First bug on Mars			

Definition of Operating System

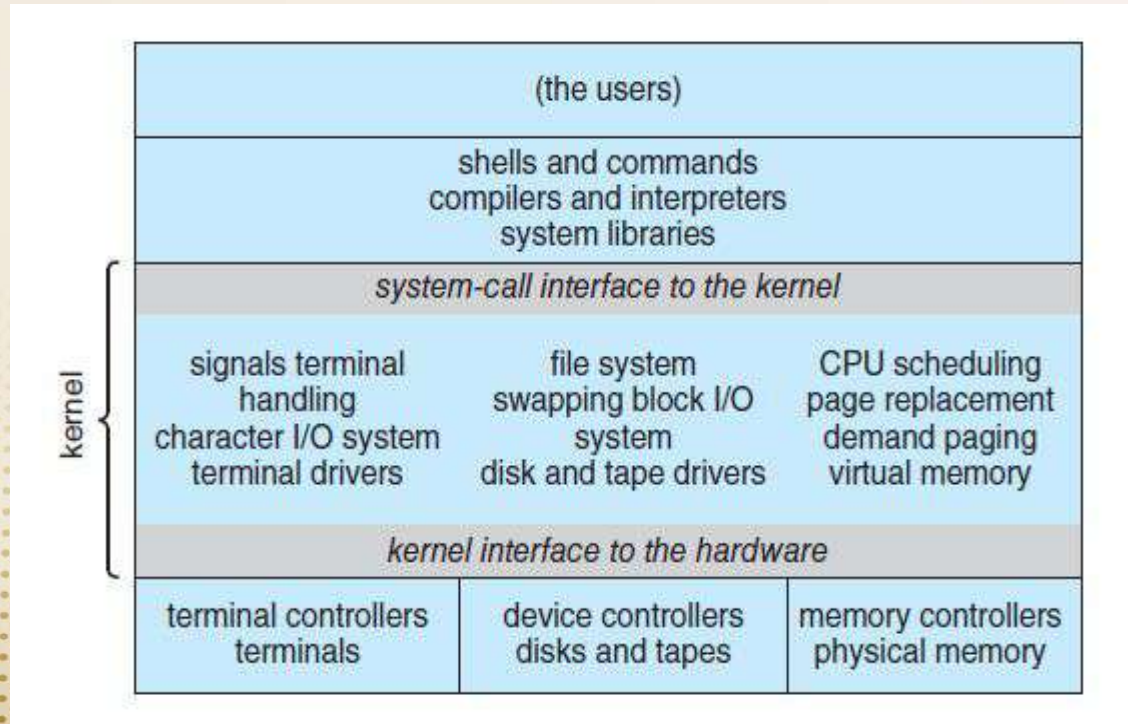
- An OS is a program that controls the execution of application programs and acts as an interface between applications and the computer hardware.
- The operating system is the one program running at all times on the computer—usually called the **kernel**.
- Other program- **System programs**, which are associated with the operating system but are not necessarily part of the kernel,
- **Application programs**, which include all programs not associated with the operation of the system.

Mobile Operating Systems

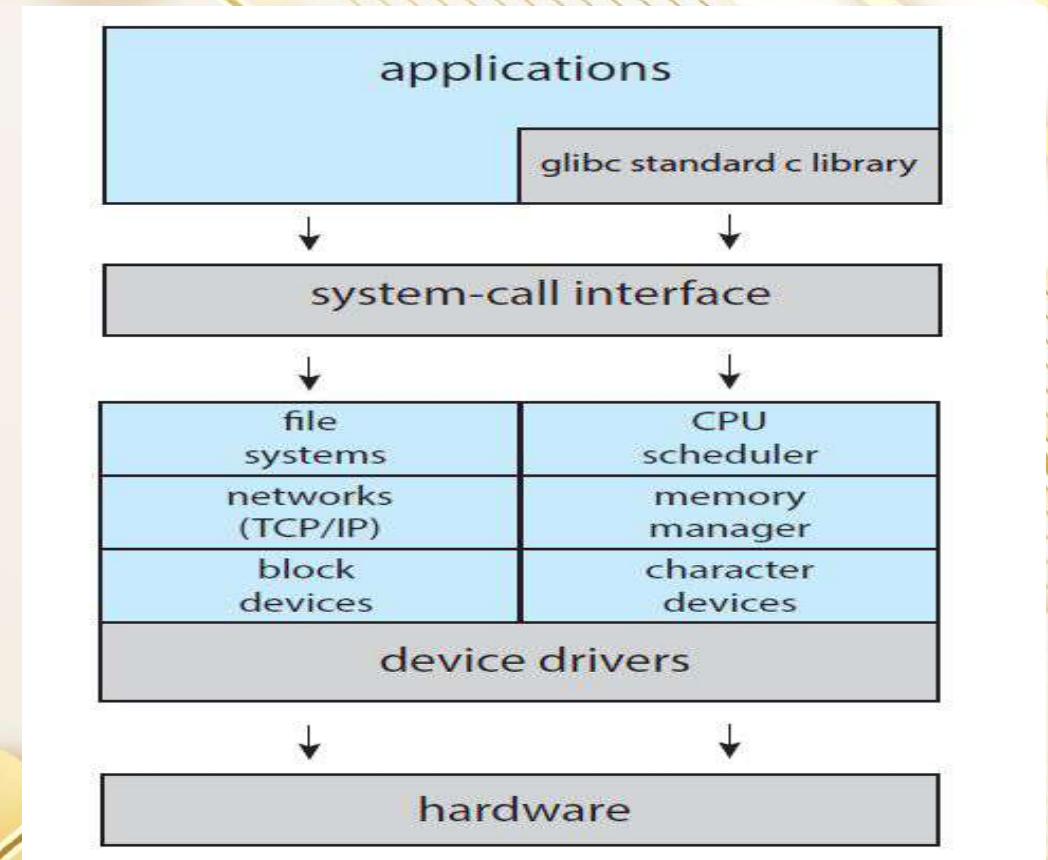
- A **core kernel** and **middleware**—a set of software frameworks that provide additional services to application developers.
- For example—Apple's iOS and Google's Android- features a core kernel along with middleware that supports databases, multimedia, and graphics.
- **Activity: Point out the differences between iOS versus Android application development.**

Operating-System Structure

Monolithic Structure: UNIX, Linux and Windows OS.



A traditional Unix operating system structure



Linux system structure.

Note: Students are advised to read: Silberschatz, Galvin, Gagne, Operating System Concepts, John Wiley and Sons, 10th edition, 2023, ISBN: 935746056X

WHY STUDY OPERATING SYSTEMS?

- OS is a key part of any computer system
 - Including embedded systems, which have OS running
- Any application needs to run on OS
- Applications use the underlying OS to control the HW below
 - For developing reliable and efficient applications, OS knowledge is a must
- Knowledge of OS is essential for Electronics engineers too because
 - CPUs are designed to have HW features which are required by the OS
 - Peripherals are designed to be interfaced with Device drivers of OS

WHY STUDY OPERATING SYSTEMS contd..?

- Linux is widespread now from the high-end servers to low power embedded systems
 - Learn OS fundamentals well, then you are ready to go deeper into Linux internals
 - OS-Lab exercises would focus on the same and provide you hands-on experience of what you learn in the theory classes
- Products in every domain has OS running in it
 - Networking equipment, Web servers, IoT, Cameras, Cars, Mobiles, etc.
- Learn OS fundamentals very well to be a successful SW engineer!!!

OS for Internet Of Things (IoT)



Courtesy: <https://devopedia.org/images/article/15/4483.1534995967.png>

Activity: Find out the types of OS

- Mainframe Operating Systems
- Server Operating Systems
- Multiprocessor Operating Systems
- Personal Computer Operating Systems
- Handheld Computer Operating Systems
- Embedded Operating Systems
- Sensor-Node Operating Systems
- Real-Time Operating Systems
- Smart Card Operating Systems

List out: Mainframe Operating Systems

- High end operating systems for mainframes (major corporate data centers).
- Mainframes are also making something of a comeback as high-end Web servers, servers for large-scale electronic commerce sites, and servers for business-to-business transactions.
- OS for mainframes are heavily oriented toward processing many jobs at once, most of which need prodigious amounts of I/O.
- They typically offer three kinds of services: batch, transaction processing, and timesharing.
- A **batch system** is one that processes routine jobs without any interactive user present.

Example can be: Claims processing in an insurance company or sales reporting for a chain of stores is typically done in batch mode.



List out Mainframe Operating Systems

- **Transaction-processing systems** handle large numbers of small requests.
- **For example**, check processing at a bank or airline reservations. Each unit of work is small, but the system must handle hundreds or thousands per second.
- **Timesharing systems** allow multiple remote users to run jobs on the computer at once, such as querying a big database. These functions are closely related; mainframe operating systems often perform all of them.
- **For example**, mainframe operating system is OS/390, a descendant of OS/360. However, mainframe operating systems are gradually being replaced by UNIX variants such as Linux.

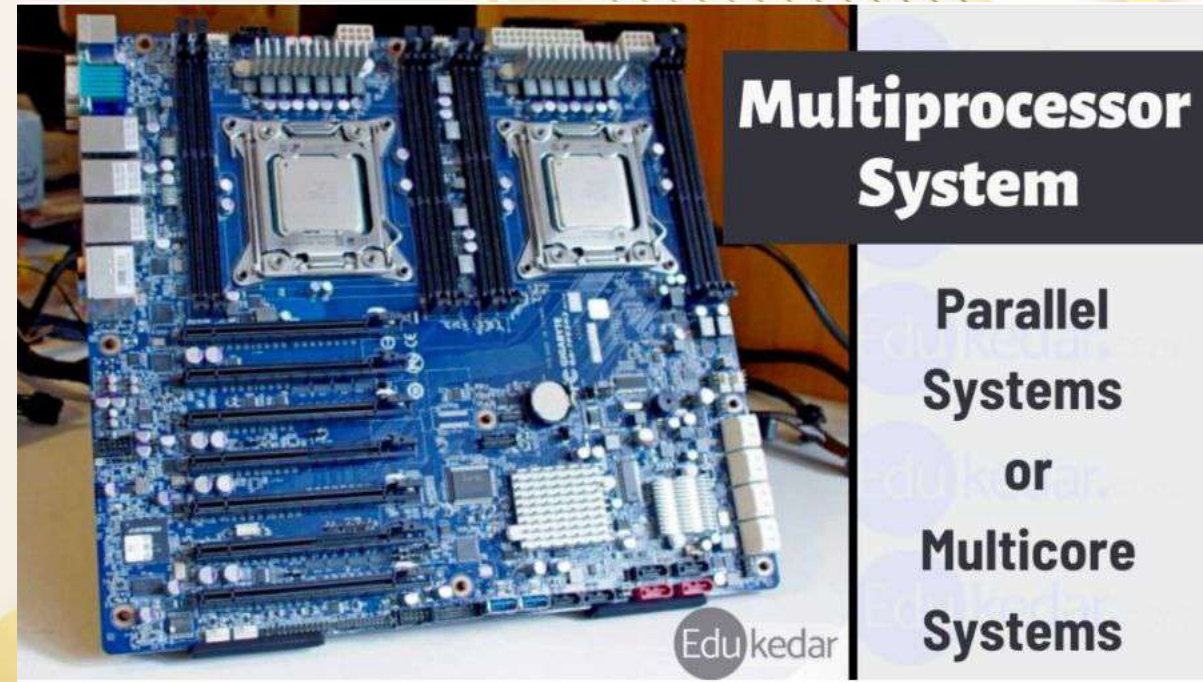
List out Server Operating Systems

- One level down are the server operating systems. They run on servers, which are either very **large personal computers, workstations, or even mainframes**.
- Serve multiple users at once over a network and allow the users to share hardware and software resources.
- **For example**, servers can provide print service, file service, or Web service. Internet providers run many server machines to support their customers and Websites use servers to store the Web pages and handle the incoming requests.
- Typical examples of server operating systems are **Solaris, FreeBSD, Linux and Windows Server 201x**.



List out Multiprocessor Operating Systems

- Connecting multiple CPUs into a single system to increase computing power.
- They are called parallel computers, multi-computers, or multiprocessors depending on how they are connected and what is shared.
- With the advent of multicore chips for PCs, even conventional desktop and notebook operating systems are starting to deal with at least small-scale multiprocessors and the number of cores is likely to grow over time.
- For example, many popular operating systems, including Windows and Linux, run on multiprocessors.
- Using the knowledge about multiprocessor operating systems collected from years of research in multicore systems should not be hard.
- The hard part will be having applications making use of all this computing power.



<https://edukedar.com/multiprocessor/#gsc.tab=0>

List out Personal Computer Operating Systems



- Their job is to provide good support to a single user.
- Modern ones all support multiprogramming, often with dozens of programs started up at boot time.
- They are widely used for word processing, spreadsheets, games, and Internet access.
- Common examples are Linux, FreeBSD, Windows 7, Windows 8, and Apple's OS X.



List out Handheld Computer Operating Systems

- Smaller systems, like tablets, smartphones and other handheld computers use this OS.
- A handheld computer, originally known as a PDA (Personal Digital Assistant), is a small computer that can be held in your hand during operation.
- For example, Google's Android and Apple's iOS.



List out the Embedded Operating Systems

- Embedded systems run on the computers that control devices that are not generally thought of as computers and which do not accept user-installed software.
- Examples are microwave ovens, TV sets, cars, DVD recorders, traditional phones, and MP3 players.
- The main property which distinguishes embedded systems from handhelds is the certainty that no untrusted software will ever run on it.
- We cannot download new applications to our microwave oven—all the software is in ROM. This means that there is no need for protection between applications, leading to design simplification.
- Systems such as Embedded Linux, QNX and VxWorks are popular.



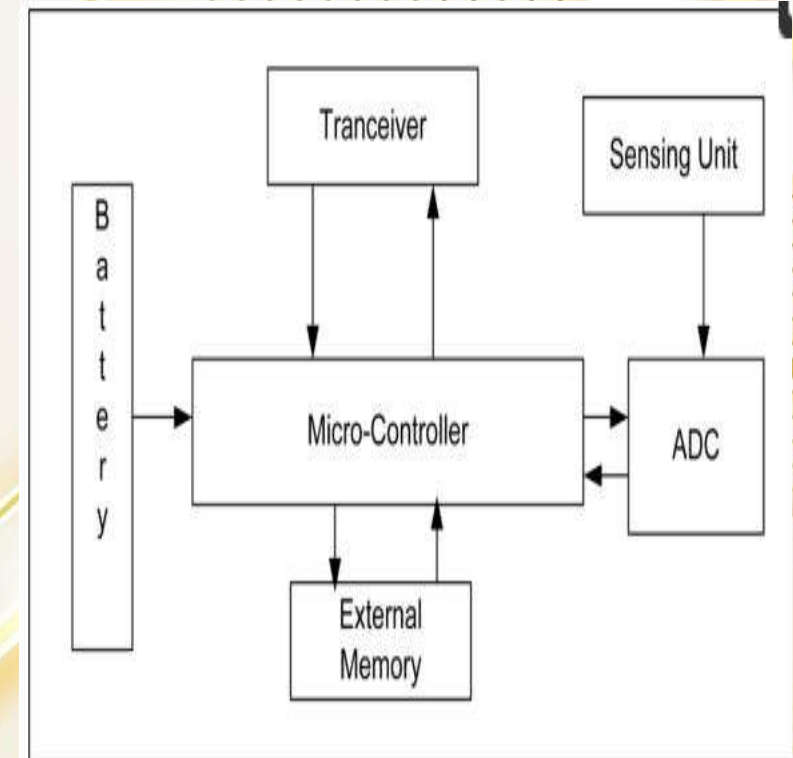
A parking meter



Air bags for cars

List out Sensor-Node Operating Systems devices

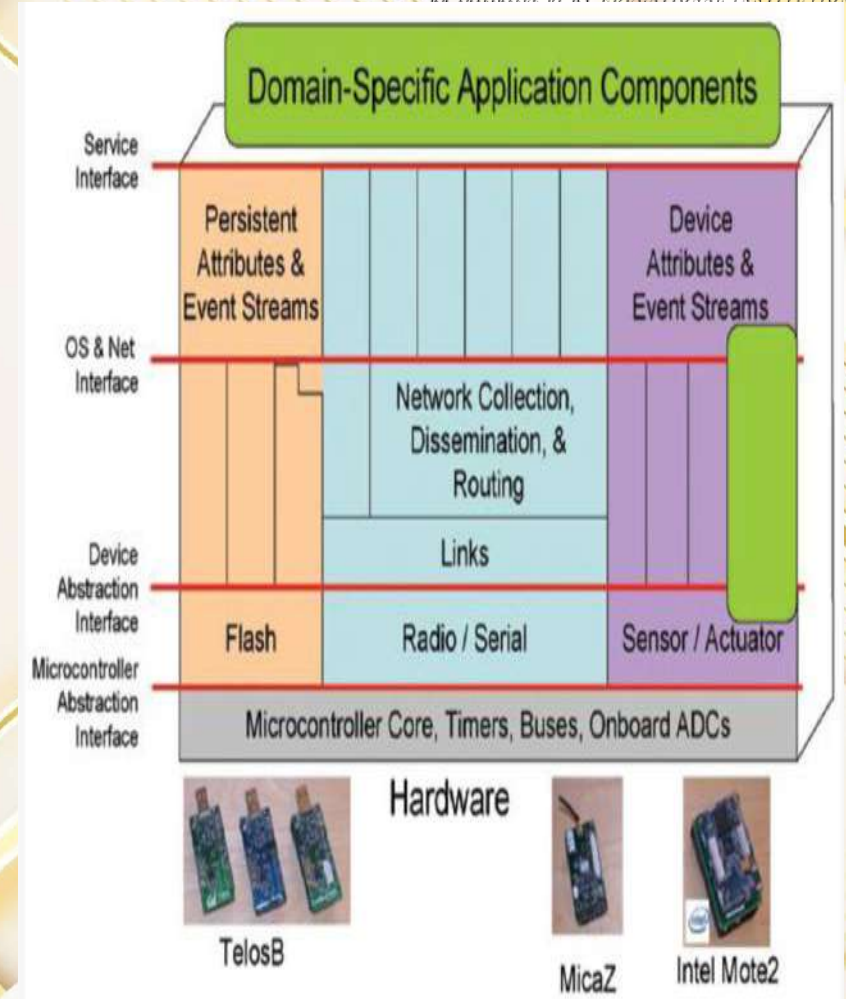
- Sensor nodes are tiny computers that communicate with each other and with a base station using wireless communication.
- Sensor networks are used to protect the perimeters of buildings, guard national borders, detect fires in forests, measure temperature and precipitation for weather forecasting, glean information about enemy movements on battlefields, and much more.
- The sensors are small battery-powered computers with built-in radios. They have limited power and must work for long periods of time unattended outdoors, frequently in environmentally harsh conditions.
- The network must be robust enough to tolerate failures of individual nodes, which happen with ever-increasing frequency as the batteries begin to run down.



Sensor Node Architecture

Sensor-Node Operating Systems

- Every sensor runs a small, but real and event driven OS, responding to external events or making measurements periodically based on an internal clock.
- The operating system has to be small and simple because the nodes have little RAM and battery lifetime possess a major issue.
- All the programs are loaded in advance; users do not suddenly start programs they downloaded from the Internet, which makes the design much simpler.
- For example, TinyOS is a well-known operating system for a sensor node. Other examples are: Contiki, RIOT, Mantis, and FreeRTOS used on devices like Telos, MICAz, ez430-RF2500, and WSN430 sensor nodes from manufacturers like University of California, Berkeley, MEMSIC, Texas Instruments, and FIT IoT-Lab.
- Features: offering features like power efficiency and event-driven programming for low-power sensor networks.



TinyOS Architecture

Find out types of Real time Operating Systems with examples

- Time as a key parameter.
- ROS are of two kinds.
- **Hard real-time systems:** These systems must provide absolute guarantees that a certain action will occur by a certain time.
- In industrial process-control systems, real-time computers have to collect data about the production process and use it to control machines in the factory. Often there are hard deadlines that must be met. **For example**, if a car is moving down an assembly line, certain actions must take place at certain instants of time. If, for example, a welding robot welds too early or too late, the car will be ruined.
- Many of these are found in industrial process control, avionics, military, and similar application areas.



Missile embedded with a tracking system

Real time Operating Systems

- A **Soft real-time system** is one, where missing an occasional deadline while not desirable, is acceptable and does not cause any permanent damage.
- Digital audio or multimedia systems fall in this category. Smartphones are also soft real-time systems.
- Real-time systems are more for industrial usage.

Find out Smart Card Operating System device

- The smallest OS runs on smart cards, which are credit-card-sized, devices containing a CPU chip.
- They have very severe processing power and memory constraints. Some are powered by contacts in the reader into which they are inserted, but contactless smart cards are inductively powered, which greatly limits what they can do.
- Some of them can handle only a single function, such as electronic payments, but others can handle multiple functions. Often these are proprietary systems.

Smart Card Operating Systems

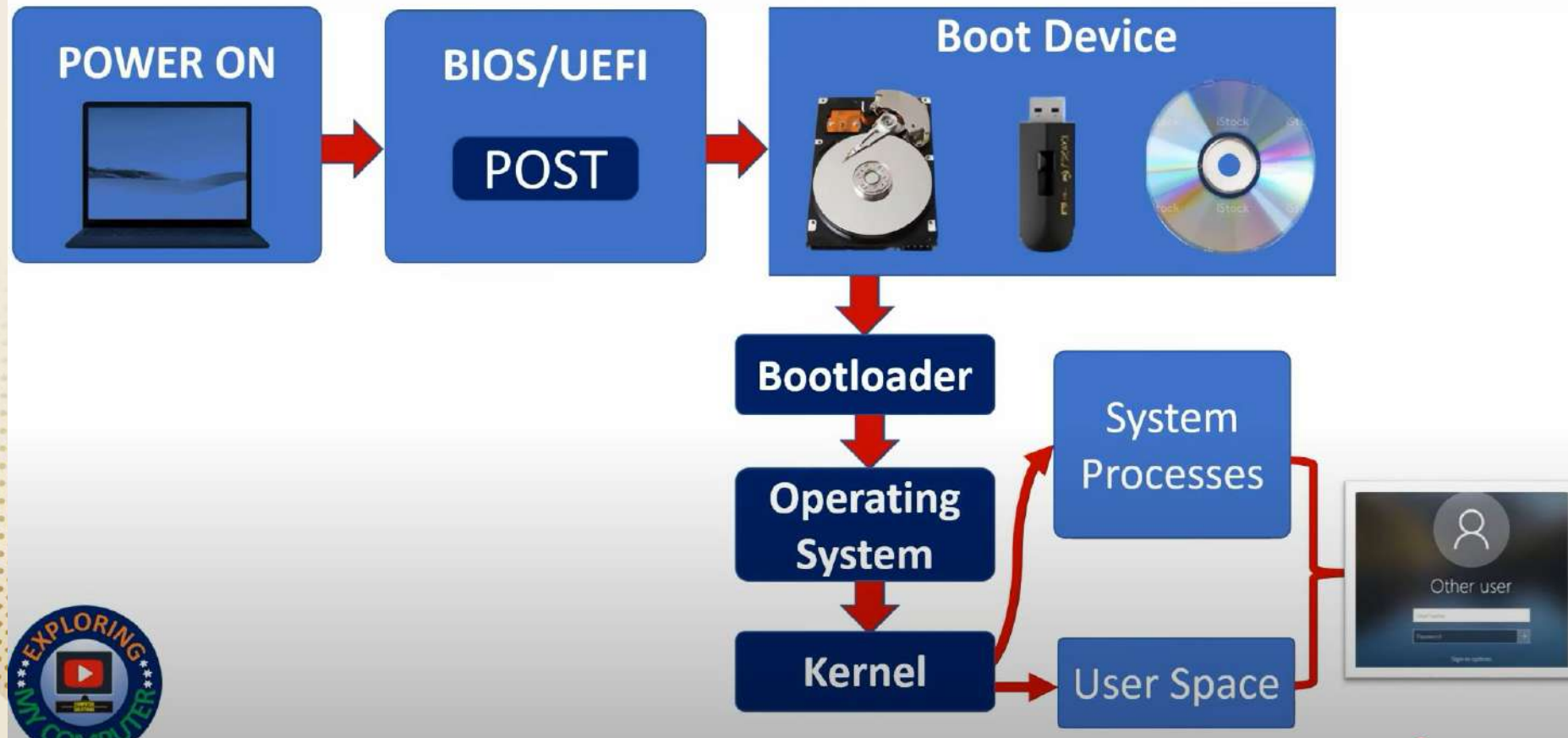
- Some **smart cards are Java oriented**. The ROM on the smart card holds an interpreter for the Java Virtual Machine (JVM). Java applets (small programs) are downloaded to the card and are interpreted by the JVM interpreter.
- Some of these cards can handle multiple Java applets at the same time, leading to multiprogramming and the need to schedule them. Resource management and protection also become an issue when two or more applets are present at the sametime. These issues must be handled by the (usually extremely primitive) operating system present on the card.



How to start/ boot a computer?

- **Start** (press On button) -> **Bootstrap program** (runs an initial program, which initializes CPU registers, device controllers, memory content) -> **locates OS kernel** (core part of OS), and loads it into the memory -> **OS** then starts **executing** the first process, such as "**init**" -> and waits for some event to occur (**Interrupt**- signal to CPU requesting attention) either from h/w or s/w.
- Hardware may trigger an interrupt at any time by sending a signal to the CPU, usually by way of the system bus. Software may trigger an interrupt by executing a special operation called a system call (also called a monitor call).

The Boot Process Sequence as follows...



Refer to: <https://www.youtube.com/watch?v=Q3rssfwxcyA>

Booting the Computer

- The computer is switched off completely and if turned on then it is called cold booting. Warm booting is a process of using the operating system to restart the computer.

****Bonus annotations for more information on booting process, you may refer to****:

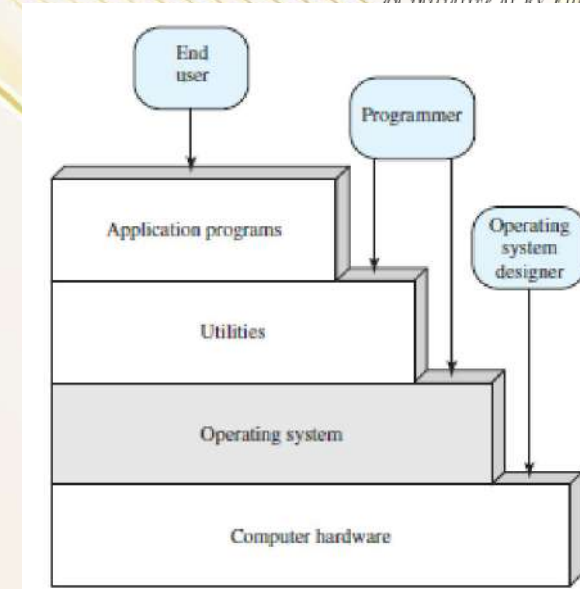
<http://kcg.digimat.in/nptel/courses/video/106106144/L11.html>

List out the functions of OS

- Booting the Computer
- Memory Management
- Processor Management
- Device Management
- File Management
- I/O Management
- Security
- Control Over System Performance
- Job Accounting
- Error-Detecting Aids
- Coordination Between Other Software and Users
- Performs Basic Computer Tasks

OS Services: program development

- OS provides a variety of facilities and services, such as editors and debuggers, to assist the programmer in creating programs.
- Typically, these services are in the form of **utility programs** that, while **not strictly part of the core of the OS**, are supplied with the OS and are referred to as **application program development tools**.
- **Example:** Visual C++ on Windows, Eclipse and code::Blocks on both Windows and Linux



On both Windows and Linux

Utilities:
Development Environment

On Windows



Visual C++

Xcode on Mac OS





eclipse



Code::Blocks

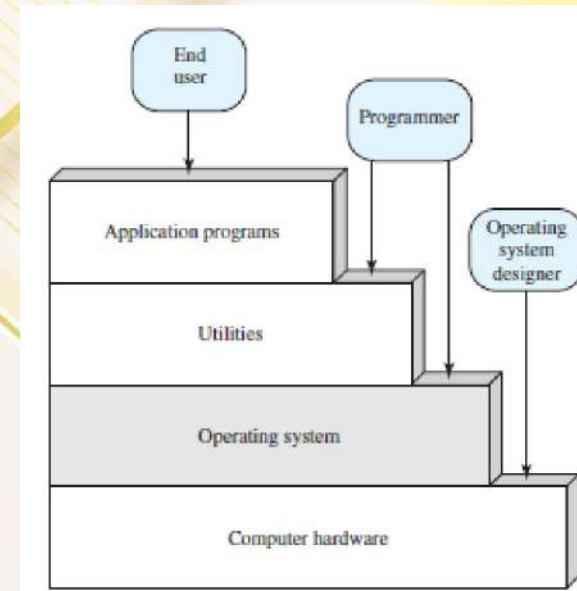
OS Services: program execution

- A number of steps need to be performed to execute a program.
- Instructions, data and stack (resident set) of the program need to be present on the main memory before the execution starts.
- I/O devices (HDD) must be initialized, and other resources (for loading files on to RAM) must be prepared.
- The OS handles these scheduling duties for the user.
- Example: Windows does all the above while this PowerPoint presentation is being shown now.

Q. What will be on Main Memory (MM) while this file is opened using PowerPoint? Show the resident sets of the programs on MM.

Windows (kernel) always present	Windows (Data structure related to this instance of PPT)	PowerPoint (Microsoft office tool)	(File opened from HDD loaded into MM)	
---------------------------------	--	------------------------------------	---------------------------------------	--

← Main Memory (DRAM)



Q. What is a **resident set** of a program?

Ans: The required contents of a program, for it to run: Code + Data + Stack

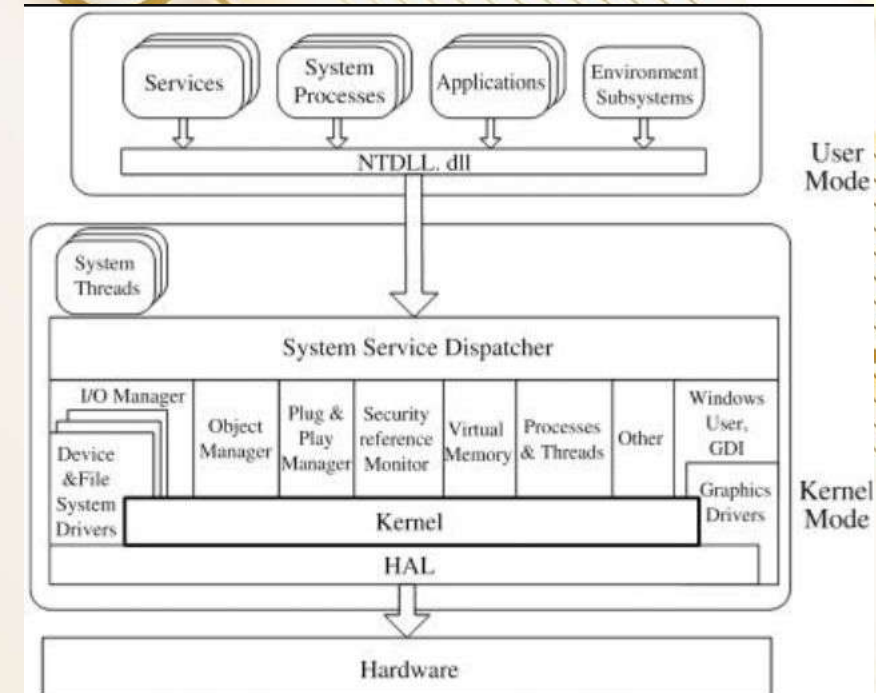
Note: Each **box** is a **Resident set** of a **program**. It means, code, data, stack of the program are all included.

OS Services: Access to I/O Devices

- Hardware Abstraction Layer (HAL) and Device drivers are I/O devices (HW) related programs, which are bundled with the OS
 - They help the application programs to interact with the HW (read from and write into it, or control it)
- Each I/O device requires its own peculiar set of instructions or control signals for its operation.

Both **Serial port** and **File system** have **Open(), read(), write()**

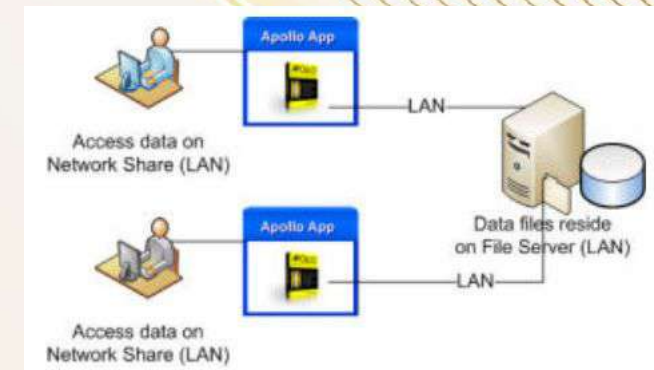
- I/O devices include:
 - Keyboard, touch pad, mouse, HDD, display, etc.
- OS provides a uniform interface that hides these details so that programmers can access such devices using simple reads and writes



HAL: It abstracts the access of different HW peripherals with a common set of APIs.

OS Services: Controlled Access to Files

- For file access, the OS must have a detailed understanding of not only the nature of the I/O device (disk drive, tape drive)
- But also the file system structure and how a file is stored on the storage medium (HDD).
- Note that, Session2 PPT file contents and its structure are known to the PowerPoint program and but need not be known to the OS.
- OS reads the PPT file as stream of bytes and loads it into the memory and the PowerPoint program understands the contents.
- In the case of a system with multiple users, the OS may provide protection mechanisms to have a controlled access (read only) to the files.

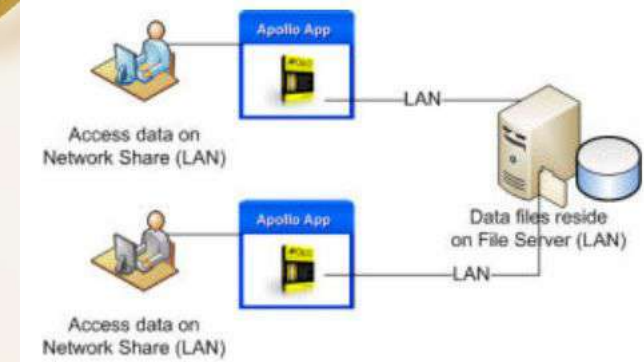


Example: Instructor reloading a file which is already posted on Moodle, while the students are also accessing it to download it from Moodle.

Note: Try to understand the overall context here and the role played by the OS. Now, do not worry about how they are implemented in the OS.

OS Services: System Access

- For shared or public systems, the OS controls the access to the system as a whole and also to the specific system resources (files).
 - User authentication and password to login into the laptop (controlled system access).
 - Password protection while the file server is accessed over the network by the users.
- The access function must also provide protection of resources and data from unauthorized users and must resolve conflicts for resource contention.
 - **Restricted access** to certain files based on the user.
 - Not allowing a file to be opened for updates while the same file is being updated by another user.

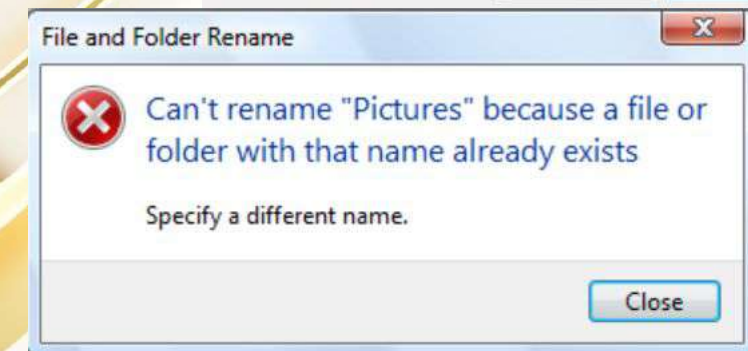


Note: While the PowerPoint executable is accessed for running, it should be accessed as read-only. It's contents shouldn't be modified while accessing it to run a PowerPoint instance.

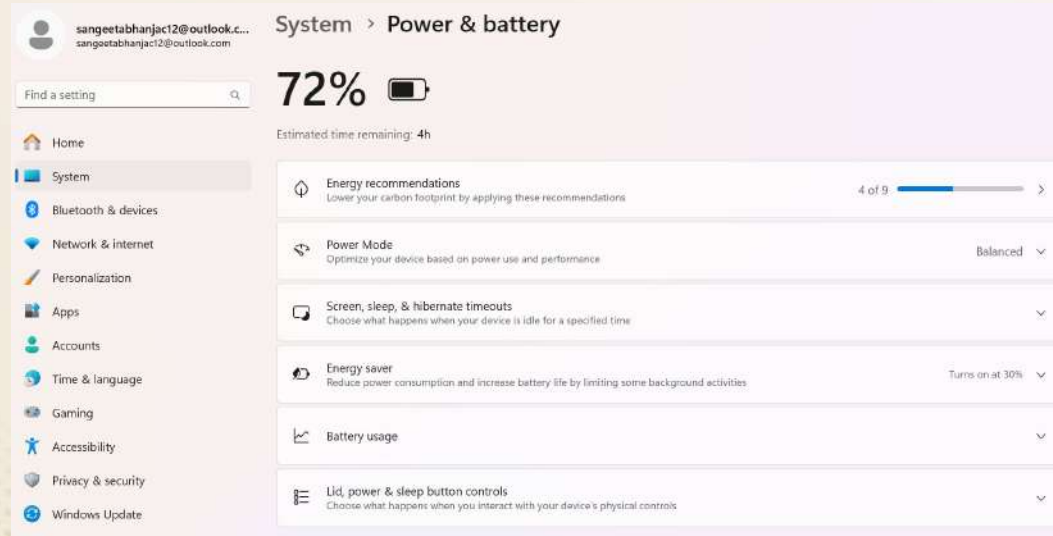
Note: Whereas the ppt file used here is not read-only, user should be able to write into it.

OS Services: Error Detection and Response

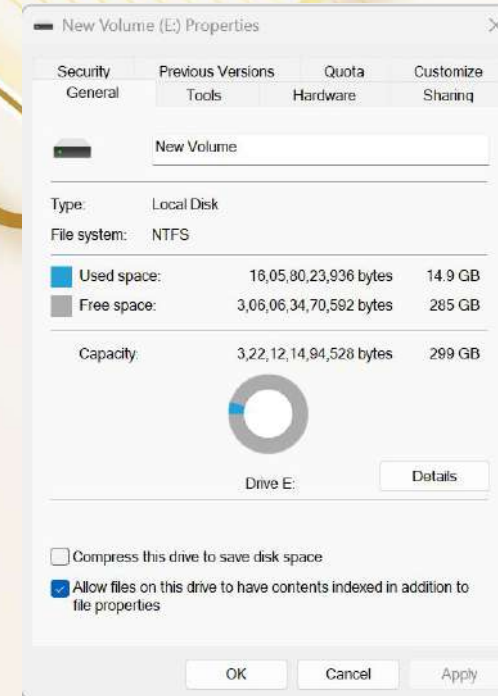
- A variety of errors can occur while a computer system is running.
- These include internal and external hardware errors, such as a device failure or malfunction.
- Various software errors, such as division by zero, attempt to access forbidden memory location due to corrupted pointers, or a drive which does not exist etc.
- File access violation error, due to a missing file or not having an access rights to it, etc.
- Preventing erroneous action by the user.
- Renaming a file, same as an already existing file with the same name is prevented by the OS.



OS Services: Accounting



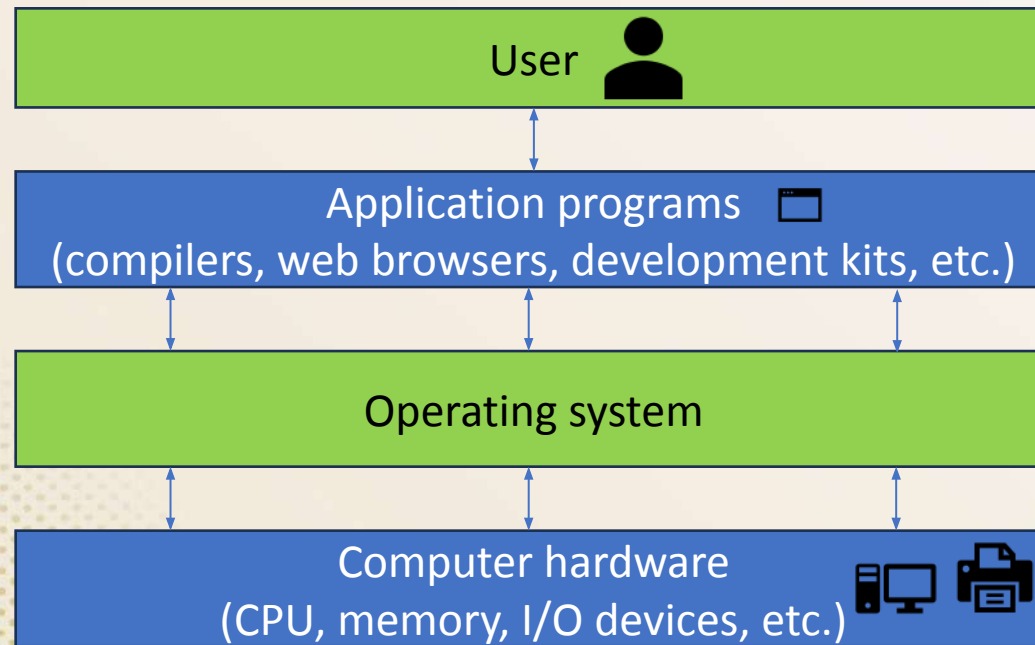
Battery Health



Disc utilization

- A good OS will collect usage statistics for various resources and monitor performance parameters such as response time.
- On any system, this information is useful in anticipating the need for future enhancements and in tuning the system to improve performance.
- On a multiuser system, accounting information can be used for billing purposes.
- Example: Disk usage, battery health, CPU utilization, network bandwidth usage, etc.

Abstract View of the Components of a Computer System.



Users'
view

Based on interface what they use

Abstract View of the Components of a Computer System.

- **Resource allocator**- in terms of CPU time, memory space, storage space, I/O devices, and so on.
- **Conflict solver** between competing resources.
- **A control program**: manages the execution of user programs to prevent errors and improper use of the computer. It is especially concerned with the operation and control of I/O devices.

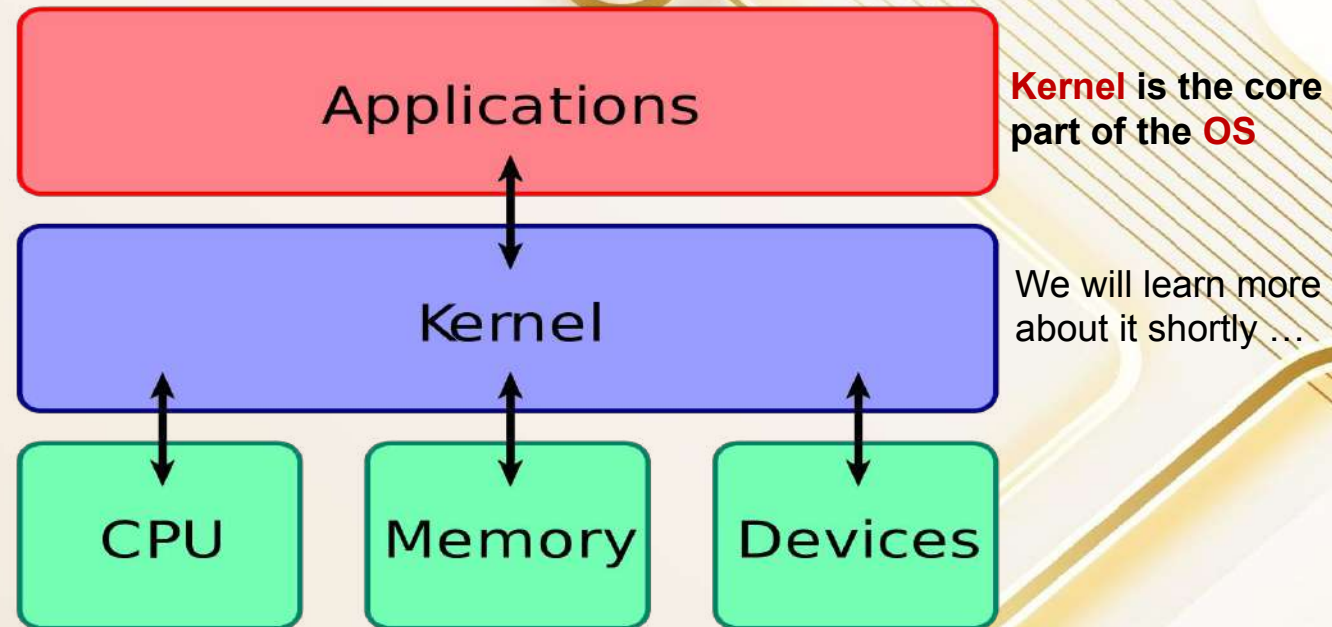
System
view

How is an Executable loaded into the DRAM?

An executable gets loaded into RAM in a process called "loading." The operating system reads the contents of the executable file from storage (such as a hard drive or flash drive) and loads it into a region of memory called the "process memory space." The whole file does not necessarily get loaded into RAM at once. The operating system uses a technique called "demand paging" to load only the parts of the file that are needed for the program to run. This allows for more efficient use of memory, as the parts of the file that are not needed are not loaded into RAM. When the program needs to access a part of the file that is not currently in RAM, the operating system will "page in" the necessary section from storage into RAM. This process can happen dynamically as the program is running, and the operating system will continue to manage the memory usage to ensure that the program has enough memory to run effectively.

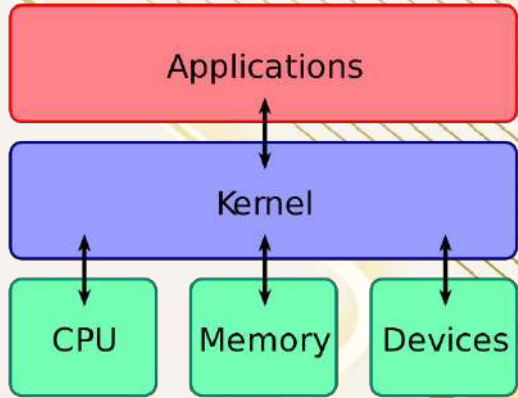
Courtesy:

What is a kernel of an OS?



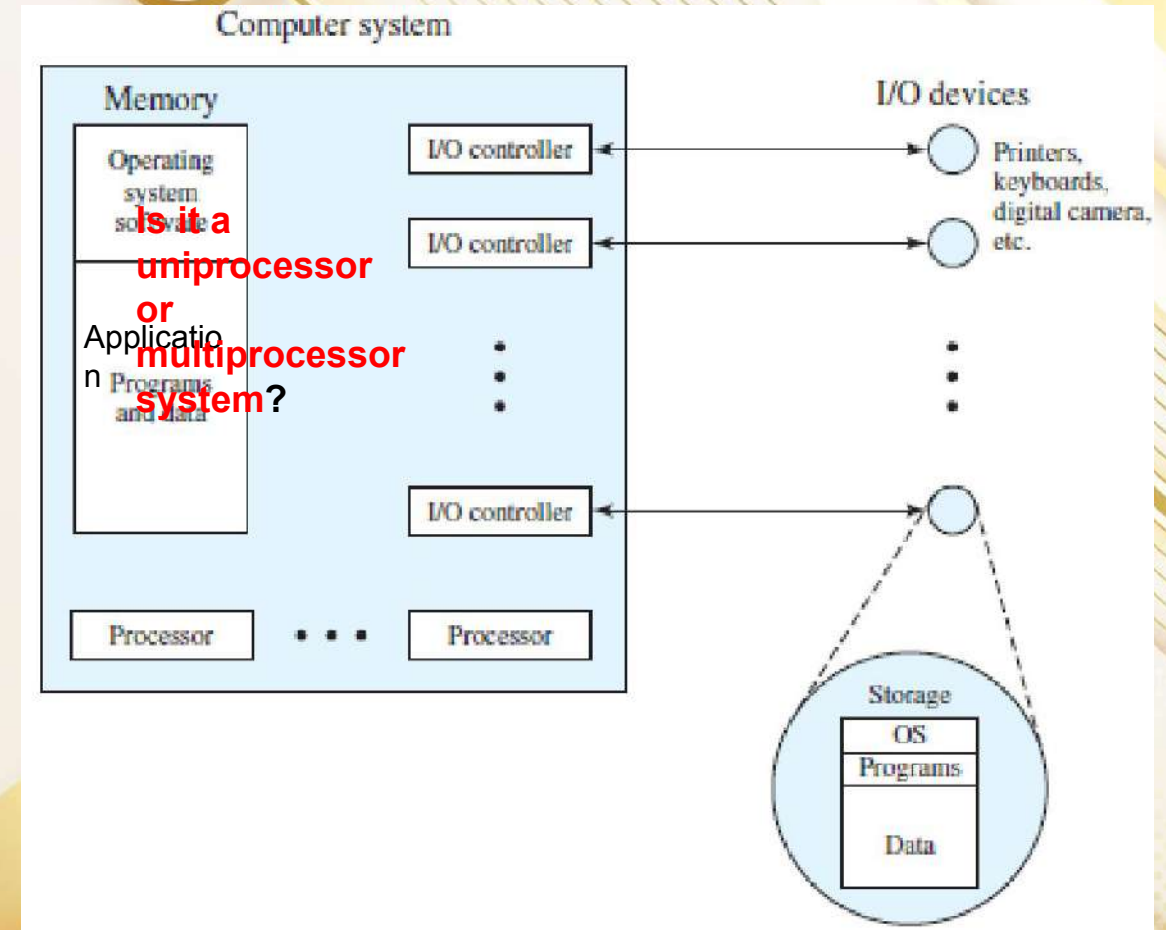
- A **kernel** is the **central part** of an **operating system**.
- It manages the operations of the computer and the hardware - most notably **memory** and **CPU time**
- Managing the CPU time means that OS choosing which process to run on the CPU next (scheduler) and how long.

Time-sharing of Resources

- The processor itself is a resource, and the OS must determine how much processor time is to be devoted to the execution of a particular user program.
 - In the case of a multiple-processor system, this decision must span all of the processors.
- 
- The diagram illustrates the interaction between user applications and system resources. At the top is a red box labeled 'Applications'. Below it is a blue box labeled 'Kernel'. At the bottom are three green boxes labeled 'CPU', 'Memory', and 'Devices'. Bidirectional arrows connect 'Applications' to 'Kernel', and 'Kernel' to each of the three resource boxes ('CPU', 'Memory', and 'Devices').
- Though kernel is shown above the memory in the picture above, the kernel itself is a program which resides in the memory and executed by the CPU.
 - When the kernel executes (scheduler) it takes a decision about, which user process would run on the CPU next, and for how long.
 - After the stipulated time, the CPU is forcibly stopped from executing the user program and the OS takes the control back, so that it can decide which user program should run on the CPU next.
 - Effectively user programs **time-share** the CPU, memory and devices.

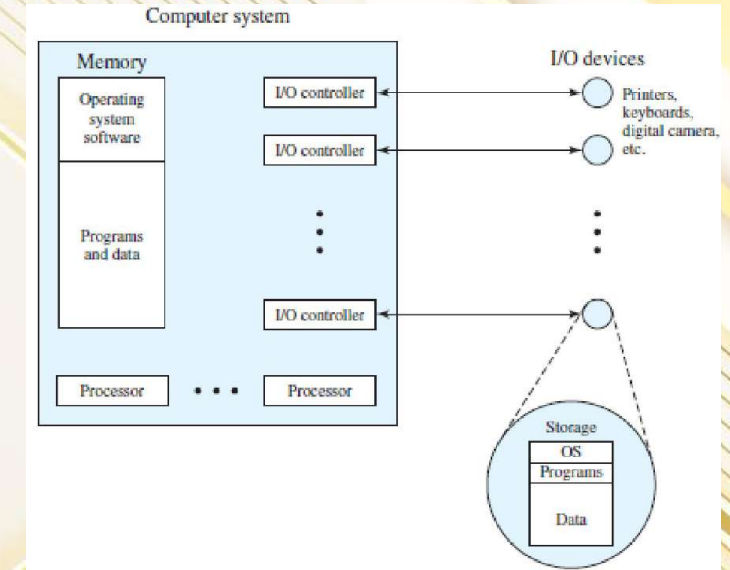
Resource Manager

- A computer is a set of resources for the movement, storage, and processing of data and for the control of these functions.
- The OS is responsible for managing these resources.
- By managing the computer's resources, the OS is in control of the computer's basic functions.
- Normally, we think of a control mechanism as something external to that which is controlled, or at least as something that is a distinct and separate part of that which is controlled.



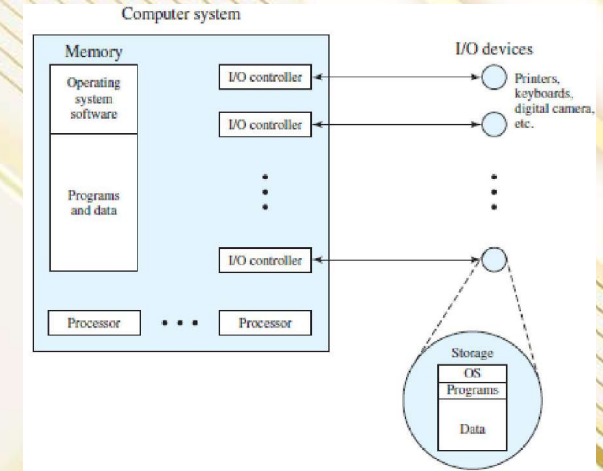
How Does the OS manage the Resources?

- For example, a residential heating system is controlled by a thermostat, which is separate from the heat-generation and heat distribution apparatus.
- This is not the case with the OS, which as a control mechanism is unusual in two respects:
 1. The OS functions in the same way as ordinary computer software; that is, it is a program or suite of programs executed by the processor.
 2. The OS frequently relinquishes control and must depend on the processor to allow it to regain control.
- The OS directs the processor in the use of the other system resources and in the timing of its execution of other programs.

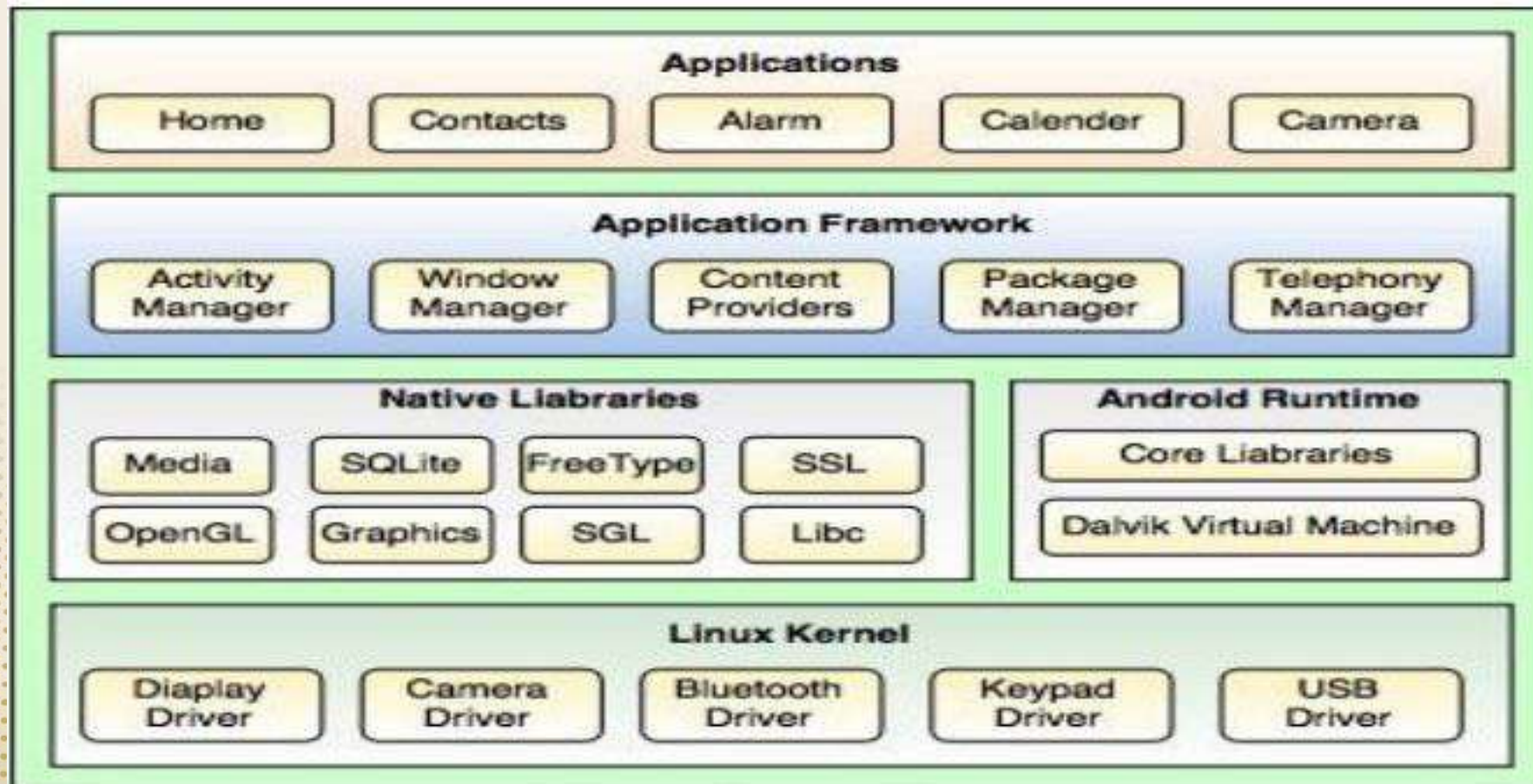


How Does the OS manage the Resources? Contd...

- But in order for the processor to do any of these things, it must cease executing the OS program and allow the CPU to execute other programs.
- Thus, the OS relinquishes control of the processor to perform some “useful” work and then resumes control long enough to prepare the processor to do the next piece of work.
- The **kernel** contains the most frequently used functions of the OS.
- The remainder of main memory contains user programs and data.
- The allocation of this resource (main memory) is controlled jointly by the OS and memory management hardware in the processor.
- The OS decides when an I/O device can be used by a program in execution and controls access to and use of files.



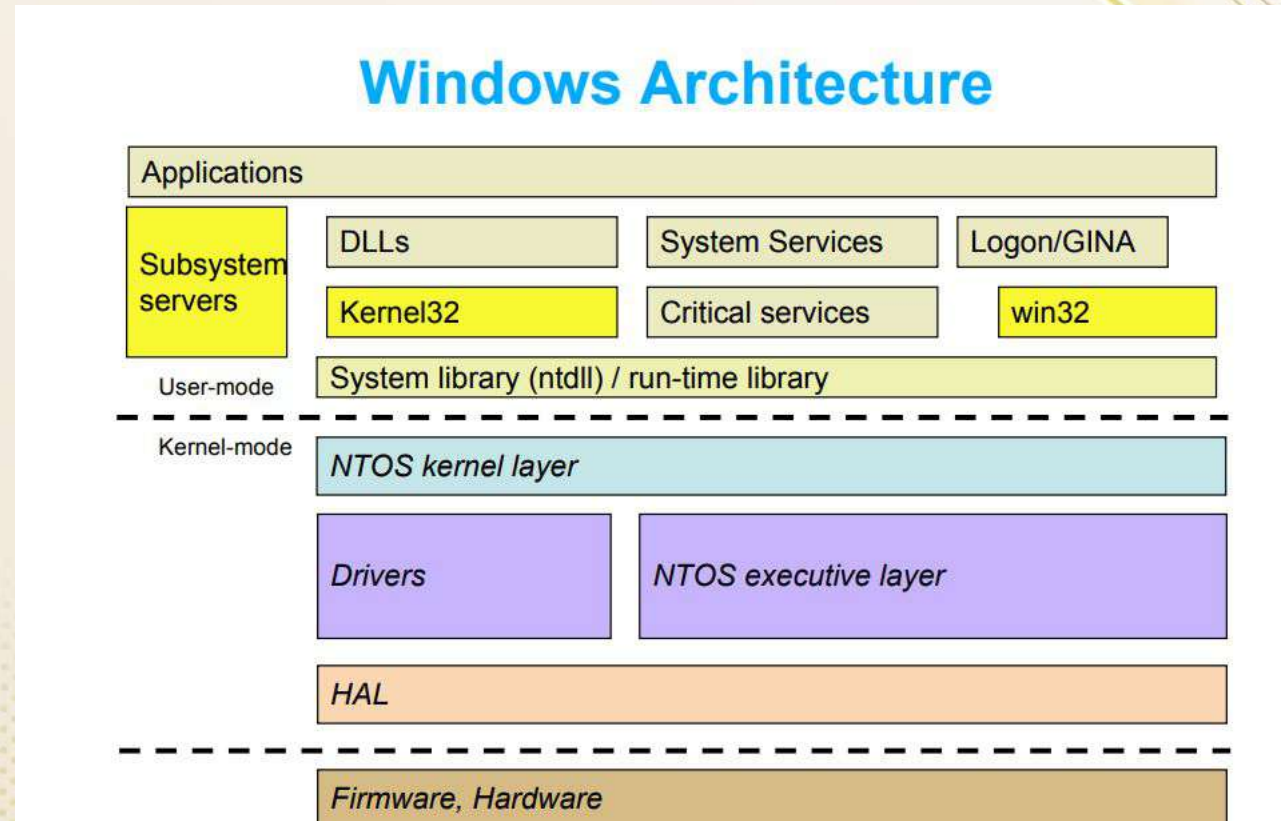
Android Architecture Representation



Refer for more details: <https://www.pcloudy.com/android-and-ios-basics-and-comparison/>

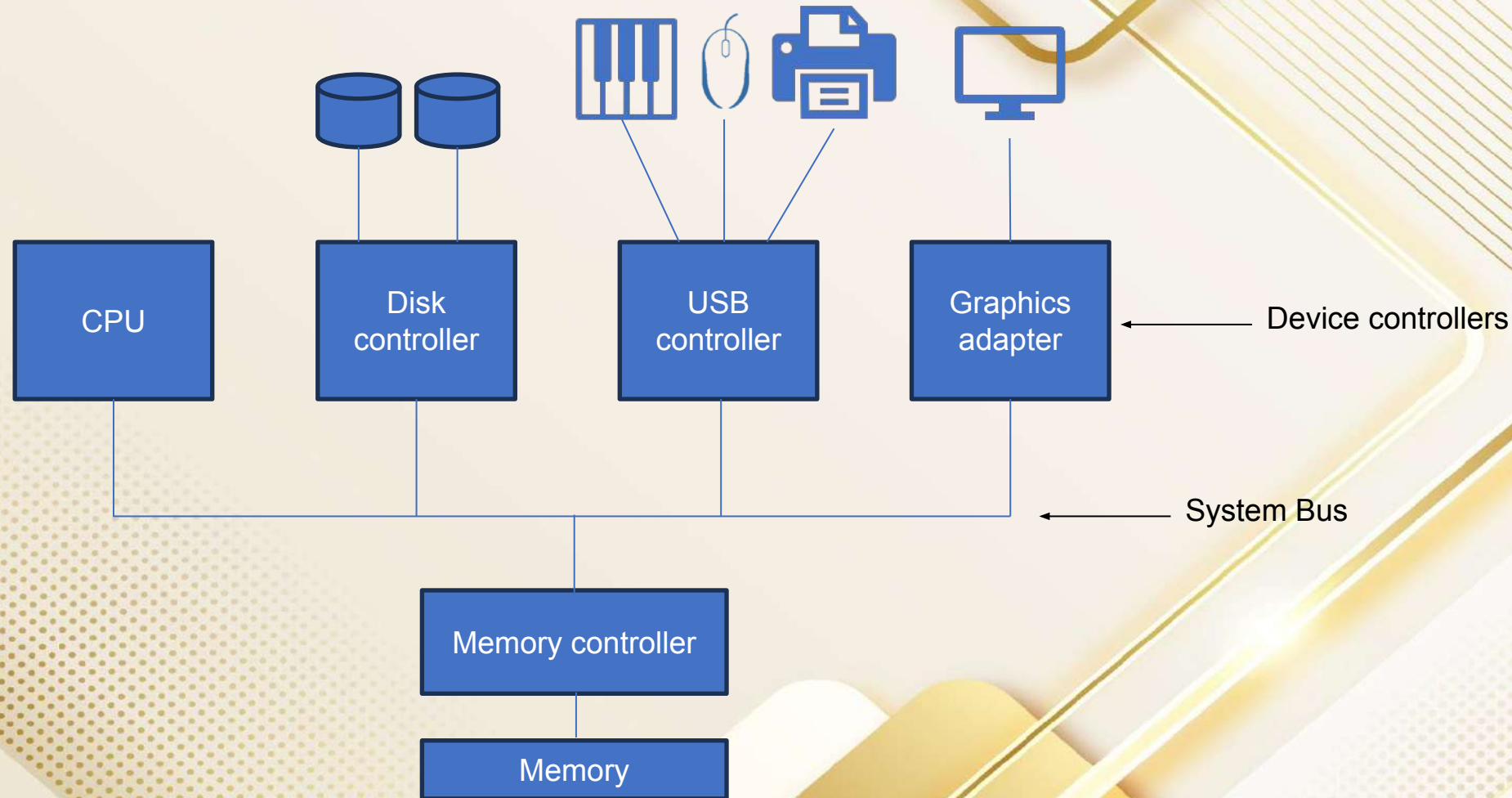
Source: <https://dotnettricksweb.blob.core.windows.net/img/xamarin/ios-architecture.png>

Windows Architecture Representation



© Microsoft Corporation 2008

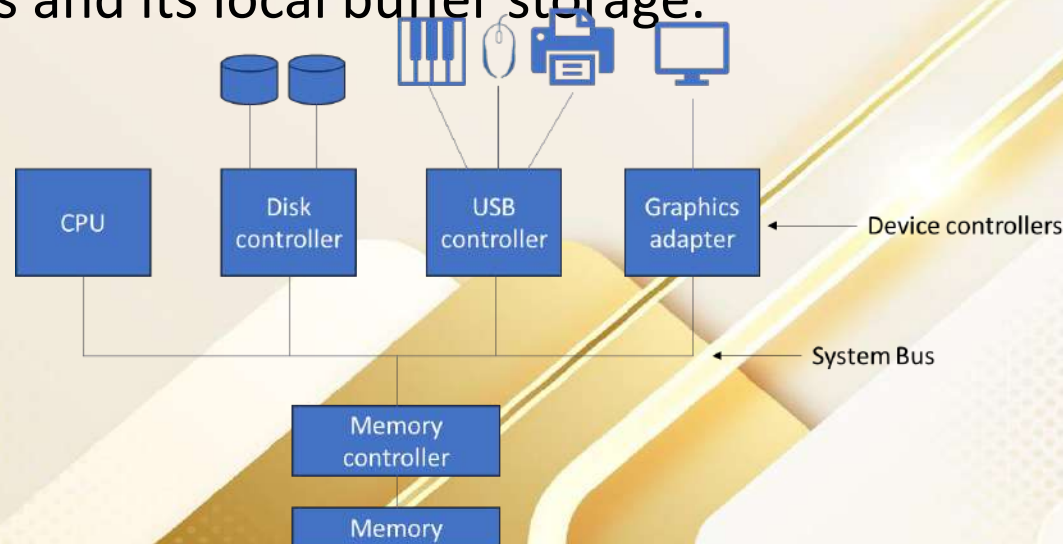
Computer-System Organization



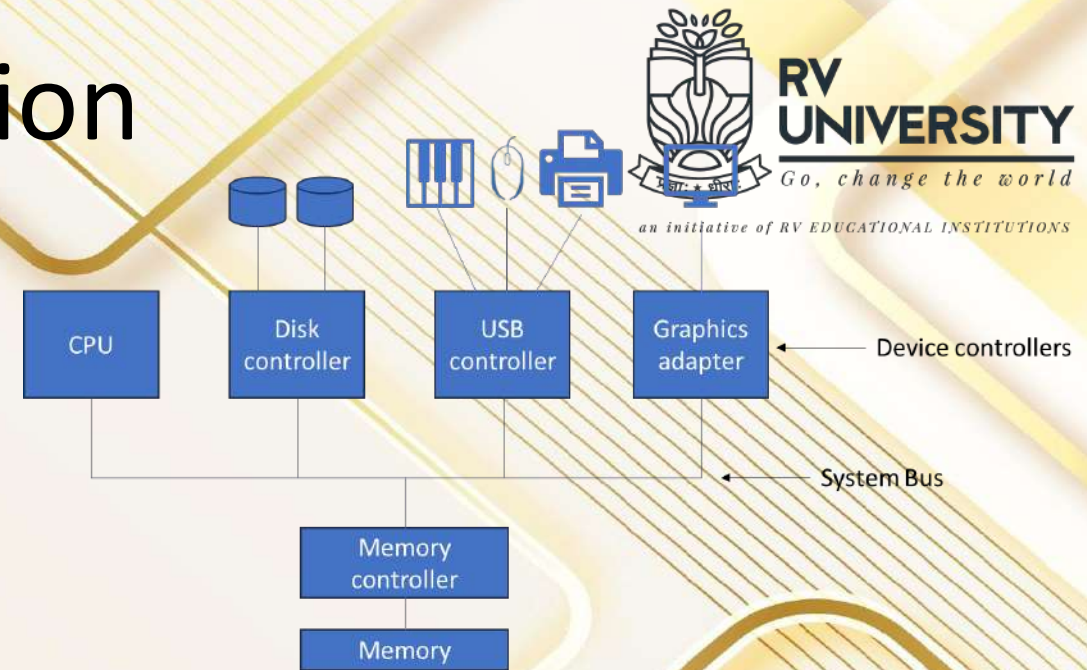
A modern general-purpose computer system consists of one or more CPUs and a number of device controllers connected through a common **bus** that provides access between components and shared memory

Computer-System Organization

- Depending on the controller, more than one device may be attached. For instance, one system USB port can connect to a USB hub, to which several devices can connect.
- A device controller maintains some local buffer storage and a set of special-purpose registers.
- The device controller is responsible for moving the data between the peripheral devices that it controls and its local buffer storage.



Computer-System Organization



- Operating systems have a device driver for each device controller.
- This device driver understands the device controller and provides the rest of the operating system with a uniform interface to the device.
- The CPU and the device controllers can execute in parallel, competing for memory cycles.
- To ensure orderly access to the shared memory, a memory controller synchronizes access to the memory.

What is Virtualization and Virtual machines?

- A technology that allows abstracting the hardware of a single computer (the CPU, memory, disk drives, network interface cards..) into several different execution environments, thereby creating the illusion that each separate environment is running on its own private computer.
- These environments can be viewed as different individual operating systems (for example, Windows and UNIX) that may be running at the same time and may interact with each other.
- The user of a virtual machine can switch among the various operating systems in the same way a user can switch among the various processes running concurrently in a single operating system.
- Virtualization allows operating systems to run as applications within other operating systems.

Demo on: create a virtual machine on your laptop. (VMware workstation)

Virtualization and Virtual machines contd..

- Virtualization software is one member of a class that also includes emulation.
- Emulation, involves simulating computer hardware in software, is typically used when the source CPU type is different from the target CPU type.
- '**Rosetta**', provides an emulation facility for example: Apple switched from the IBM Power CPU to the Intel x86 CPU for its desktop and laptop computers, allowed applications compiled for the IBM CPU to run on the Intel CPU. **In late 2020, apple started using its own Apple silicon processors in its devices, including Macs, iPhones, iPads, and Apple Watches. Apple silicon is based on ARM architecture.**
- Evolution of Virtual Machine: IBM mainframes as a method for multiple users to run tasks concurrently. Running multiple virtual machines allowed (and still allows) many users to run tasks on a system designed for a single user.
- **Q. Problems with running multiple Microsoft Windows applications on the Intel x86 CPU??**
- **Sol:** VMware created a new virtualization technology in the form of an application that ran on Windows. That application ran one or more guest copies of Windows or other native x86 operating systems, each running its own applications.

Real time example

Computer with Single OS and Three Virtual Machines

Explanation:

1. Physical Computer:

1. Represents the actual hardware (CPU, RAM, storage, etc.).
2. Houses the host operating system.

2. Host Operating System:

1. Manages the physical hardware resources.
2. Provides a platform for running virtual machines.
3. Examples: Windows, macOS, Linux.

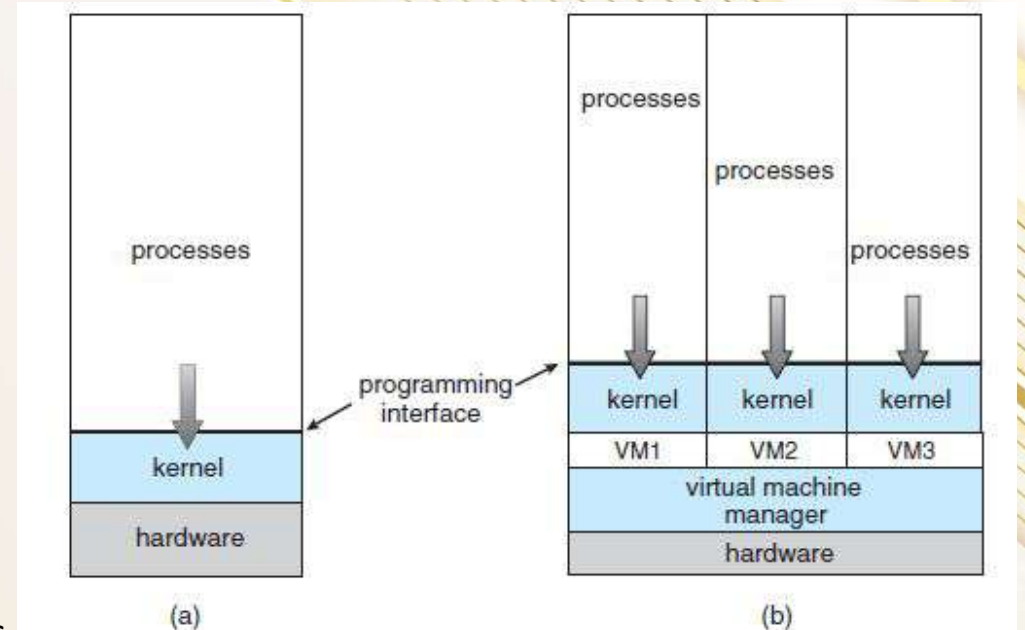
3. Hypervisor:

1. Software that creates and manages virtual machines.
2. Acts as an intermediary between the host OS and virtual machines.
3. Examples: VMware, VirtualBox, Xen.

4. Virtual Machines:

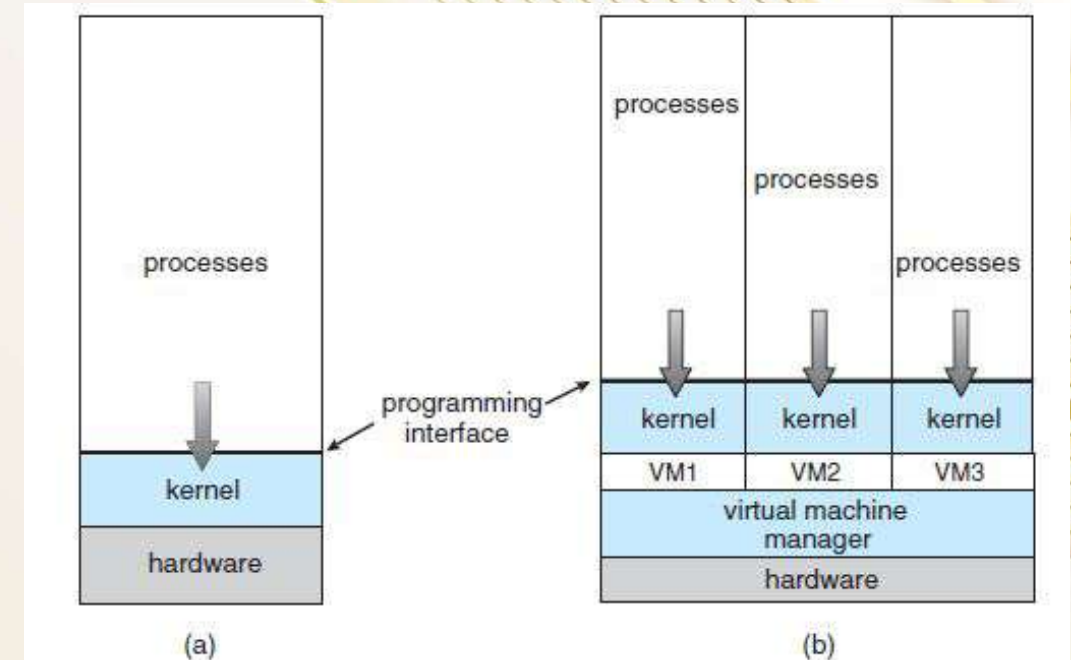
1. Software emulations of a complete computer system.
2. Each VM runs its own guest operating system.
3. Can run different OSes (e.g., Windows, Linux, macOS) simultaneously.

4. Isolated from each other and the host OS.



Key Points: Virtualization

- **Virtualization:** The process of creating virtual machines.
- **Guest OS:** The operating system running within a virtual machine.
- **Host OS:** The operating system running on the physical computer.
- **Hypervisor:** The software that enables virtualization.
- **Benefits of Virtualization:**
 - **Resource Consolidation:** Multiple OSes and applications on a single physical machine.
 - **Isolation:** VMs are isolated from each other, enhancing security.
 - **Flexibility:** Easy to create, clone, and migrate VMs.
 - **Cost-Effectiveness:** Reduced hardware costs.
 - **Testing and Development:** Ideal for testing software in different environments.

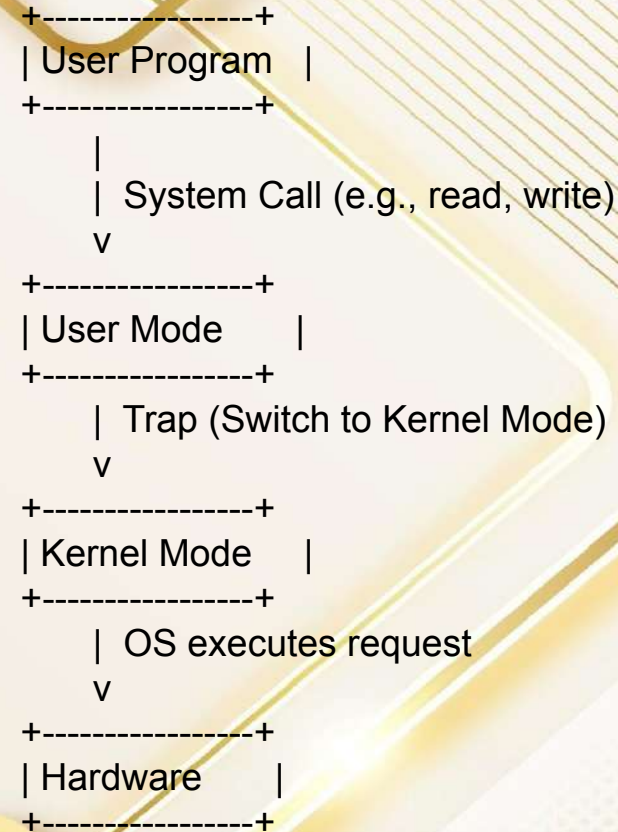


Key Concepts:

- **System Call:** A request from a user-level process to the kernel to perform a specific task.
- **Kernel Mode:** The privileged mode of operation where the kernel has full access to the system's hardware and resources.
- **User Mode:** The restricted mode of operation where user processes run.
- **File Descriptor:** An integer that represents an open file or device within a process.
- **Error Handling:** The process of detecting and responding to errors during system call execution.

System Call Workflow

1. A user program requests a system call.
2. The request is passed to the kernel via a **trap**.
3. The OS executes the requested function.
4. The result is returned to the user program.



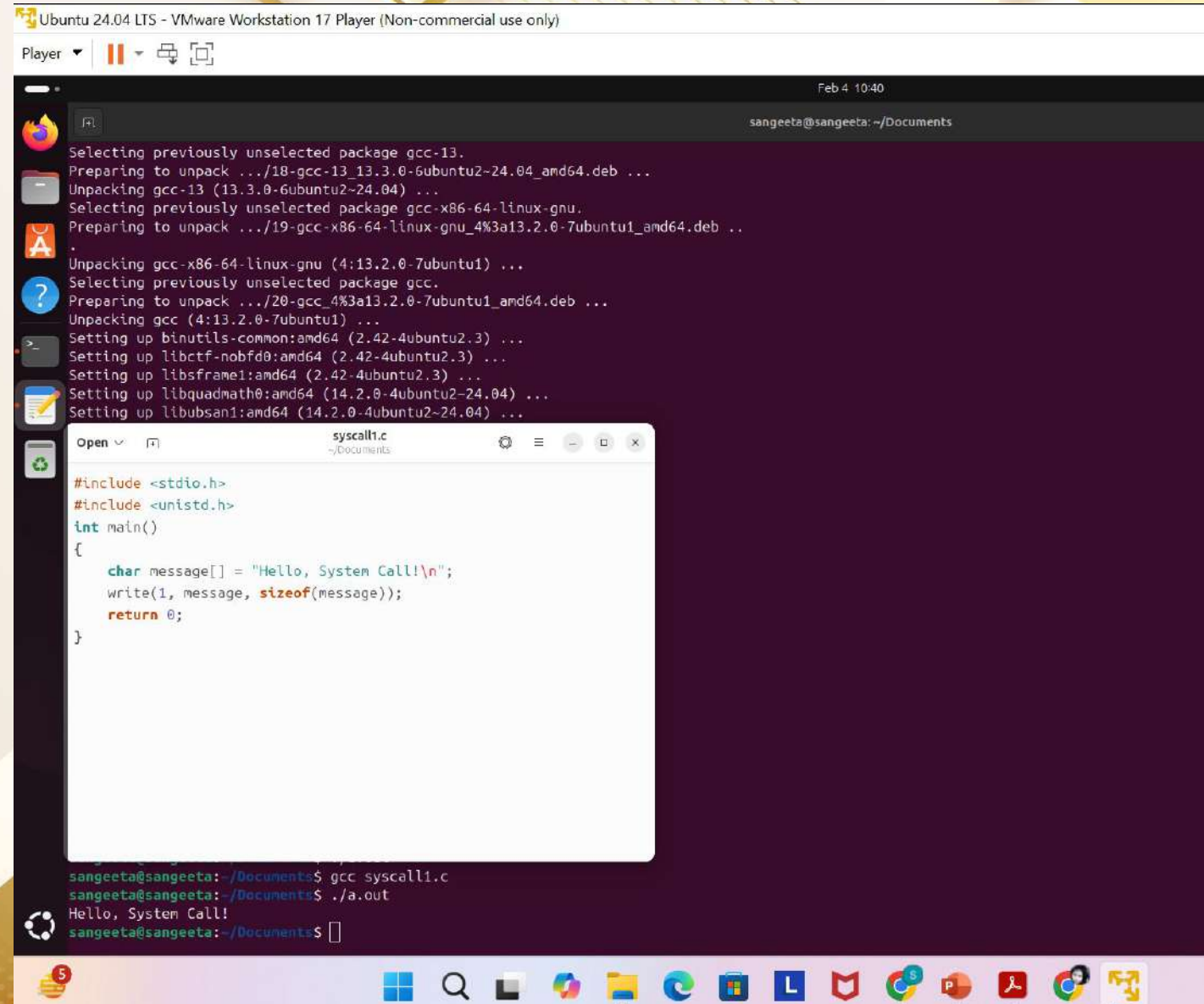
System calls: provides an interface to the services made available by OS. How???

- Writing programs using C, C++, Assembly language.
- Activity: Example of a System Call in C using write() System Call

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    char message[] = "Hello, System Call!\n";
    write(1, message, sizeof(message));
    return 0;
}
```

Explanation:

- write() system call sends data to standard output (file descriptor 1).
- It interacts with the OS kernel to display text on the screen.



The screenshot shows a terminal window titled "Ubuntu 24.04 LTS - VMware Workstation 17 Player (Non-commercial use only)". The terminal output shows the installation of GCC and the execution of a C program. The program, named "syscall1.c", is located in the "/Documents" directory. The code in the program is as follows:

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    char message[] = "Hello, System Call!\n";
    write(1, message, sizeof(message));
    return 0;
}
```

The terminal output shows the installation of GCC and the execution of the program. The output of the program is "Hello, System Call!".

write() System Call

- **write(1, message, sizeof(message));**
- The write() system call takes three parameters: “1” which is the file descriptor of the file where we want to write. Since we want to write on standard output device which is the screen, hence the file descriptor, in this case, is ‘1’, which is fixed (0 is the file descriptor for standard input device (e.g. keyboard) and 2 is for standard error device)).

Code snippet for write() system call

graph LR

```
A[User Process] --> B[System Call: write()];
B --> C[Kernel Mode];
C --> D[Check Permissions & Arguments];
D -- Valid --> E[Find File Descriptor];
E --> F[Access File/Device];
F -- Success --> G[Copy Data to Kernel Buffer];
G --> H[Write Data to File/Device];
H -- Success --> I[Update File Metadata];
I --> J[Return Status (bytes written)];
J --> K[User Process];
D -- Invalid --> L[Return Error];
L --> K;
F -- Failure --> M[Return Error];
M --> K;
H -- Failure --> N[Return Error];
N --> K;
```

Explanation of the Flow:

- 1.User Process:** The user-level application initiates the write() system call.
- 2.System Call: write():** The user process transitions to kernel mode to execute the system call.
- 3.Kernel Mode:** The kernel's system call handler for write() is invoked.
- 4.Check Permissions & Arguments:** The kernel verifies if the user process has the necessary permissions to write to the specified file/device and checks the validity of the provided arguments (file descriptor, buffer, and size).
- 5.Find File Descriptor:** If the permissions and arguments are valid, the kernel looks up the file descriptor in the process's file descriptor table to identify the corresponding file or device.
- 6.Access File/Device:** The kernel accesses the file or device associated with the file descriptor.
This might involve interacting with the file system or device driver.

graph LR

```
A[User Process] --> B[System Call: write()];
B --> C[Kernel Mode];
C --> D[Check Permissions & Arguments];
D -- Valid --> E[Find File Descriptor];
E --> F[Access File/Device];
F -- Success --> G[Copy Data to Kernel Buffer];
G --> H[Write Data to File/Device];
H -- Success --> I[Update File Metadata];
I --> J[Return Status (bytes written)];
J --> K[User Process];
D -- Invalid --> L[Return Error];
L --> K;
F -- Failure --> M[Return Error];
M --> K;
H -- Failure --> N[Return Error];
N --> K;
```

Explanation of the Flow:

7.Copy Data to Kernel Buffer: The data to be written is copied from the user process's buffer to a kernel buffer. This is a crucial step to protect the kernel from potentially malicious user data.

8.Write Data to File/Device: The kernel writes the data from its buffer to the file or device. This might involve interacting with the storage device or network interface.

9.Update File Metadata: After successfully writing the data, the kernel updates the file's metadata, such as the last modification time and file size.

10.Return Status (bytes written): The kernel returns the number of bytes actually written to the user process. This value might be less than the requested size if an error occurred or if the write operation was interrupted.

11.User Process: The user process resumes execution after the system call returns.

12.Return Error: If any error occurs during the process (e.g., invalid file descriptor, insufficient permissions, write failure), the kernel returns an appropriate error code to the user process. The user process can then handle the error accordingly. The diagram shows various points where errors can occur.


```
Feb 20 21:46 sangeeta@sangeeta: ~/Documents
```

```
sangeeta@sangeeta:~$ man strace  
sangeeta@sangeeta:~$ gcc syscall1.c  
cc1: fatal error: syscall1.c: No such file or directory  
compilation terminated.  
sangeeta@sangeeta:~$ cd Documents  
sangeeta@sangeeta:~/Documents$ man strace  
sangeeta@sangeeta:~/Documents$ gcc syscall1.c  
sangeeta@sangeeta:~/Documents$ gcc syscall1.c -o syscall1  
sangeeta@sangeeta:~/Documents$ ls  
a.out                insyscall.txt.c  
client               mem_segment.c  
Client              outsyscall.txt  
Clientside.c         semaphore  
'Client side C program.c' semaphore_prog.c  
copy1_mem_seg.c      server  
daemon_code.txt     Server  
Firstprogram.c       Serverside  
'include stdio h void main.c' Serverside.c  
in_syscall.c          'Server side C program.c'  
in_syscall.txt        syscall1  
insyscall.txt         syscall1.c  
sangeeta@sangeeta:~/Documents$ strace ./syscall1  
execve("./syscall1", ["/./syscall1"], 0x7ffe56d8a100 /* 48 vars */) = 0  
brk(NULL) = 0x644f65c50000  
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x725ef2750000  
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)  
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3  
fststat(3, {st_mode=S_IFREG|0644, st_size=64643, ...}) = 0  
mmap(NULL, 64643, PROT_READ, MAP_PRIVATE, 3, 0) = 0x725ef2740000  
close(3) = 0  
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3  
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"... , 832) = 832  
pread64(3, "\6\0\0\0\4\0\0\0@ \0\0\0\0\0\0\0\0 @ \0\0\0\0\0\0\0\0 @ \0\0\0\0\0\0\0\0"... , 784, 64) = 784  
fststat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0  
pread64(3, "\6\0\0\0\4\0\0\0@ \0\0\0\0\0\0\0\0 @ \0\0\0\0\0\0\0\0 @ \0\0\0\0\0\0\0\0"... , 784, 64) = 784  
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x725ef2400000  
mmap(0x725ef2428000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x280000) = 0x725ef2428000  
mmap(0x725ef25b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x725ef25b0000  
mmap(0x725ef25ff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x725ef25ff000  
mmap(0x725ef2605000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x725ef2605000  
close(3) = 0  
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x725ef273d000  
arch_prctl(ARCH_SET_FS, 0x725ef273d740) = 0  
set_tid_address(0x725ef273da10) = 3835
```


Tracing the system call (code)



```
arch_prctl(ARCH_SET_FS, 0x725ef273d740) = 0
set_tid_address(0x725ef273da10)      = 3835
set_robust_list(0x725ef273da20, 24)   = 0
rseq(0x725ef273e060, 0x20, 0, 0x53053053) = 0
mprotect(0x725ef25ff000, 16384, PROT_READ) = 0
mprotect(0x644f60497000, 4096, PROT_READ) = 0
mprotect(0x725ef2788000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x725ef2740000, 64643)         = 0
write(1, "Hello, System Call!\n\n0", 21Hello, System Call!
) = 21
exit_group(0)                        = ?
+++ exited with 0 +++
sangeeta@sangeeta:~/Documents$ cat syscall1.c
#include <stdio.h>
#include <unistd.h>
int main()
{
    char message[] = "Hello, System Call!\n";
    write(1, message, sizeof(message));
    return 0;
}
sangeeta@sangeeta:~/Documents$ strace -o out_write_file.txt ./syscall1
Hello, System Call!
sangeeta@sangeeta:~/Documents$
```



Tracing direct system call

strace is a diagnostic, debugging and instructional userspace utility for Linux. It is used to monitor and tamper with interactions between processes and the Linux kernel, which include system calls, signal deliveries, and changes of process state.

Ubuntu/ Debian // Ubuntu 24.04 LTS comes with strace installed

- `$sudo apt install strace` // Install strace
- `$strace ls` // To get the system call, argument, and the result of the call where ls is the command whose system call is traced.
- `$strace -c ls` // To count number of system call

sangeeta@sangeeta:~\$ cd Documents

sangeeta@sangeeta:~/Documents\$ strace -c ls

```
a.out          in_syscall.c      server
client         in_syscall.txt    Server
Client        insyscall.txt     Serverside
Clientside.c   insyscall.txt.c   Serverside.c
'Client side C program.c' mem_segment.c      'Server side C program.c'
copy1_mem_seg.c outsyscall.txt     syscall1
daemon_code.txt out_write_file.txt syscall1.c
Firstprogram.c semaphore
'include stdio h void main.c' semaphore_prog.c
```

% time	seconds	usecs/call	calls	errors	syscall
20.50	0.000182	10	18		mmap
11.37	0.000101	11	9		write
9.23	0.000082	16	5		read
8.78	0.000078	8	9		close
8.78	0.000078	15	5		mprotect
7.55	0.000067	8	8		fstat
7.09	0.000063	9	7		openat
4.39	0.000039	19	2		getdents64
3.38	0.000030	15	2	2	statfs
2.70	0.000024	8	3		brk
2.70	0.000024	12	2		ioctl
2.48	0.000022	11	2		pread64
2.36	0.000021	21	1		munmap
1.35	0.000012	6	2	2	access
1.24	0.000011	11	1		arch_prctl
1.24	0.000011	11	1		set_tid_address
1.24	0.000011	11	1		set_robust_list
1.24	0.000011	11	1		prlimit64
1.24	0.000011	11	1		getrandom
1.13	0.000010	10	1		rseq
0.00	0.000000	0	1		execve
100.00	0.000888	10	82	4	total

\$ strace -c ls

Usecs/call is a measurement of the number of microseconds it takes to complete a system call.

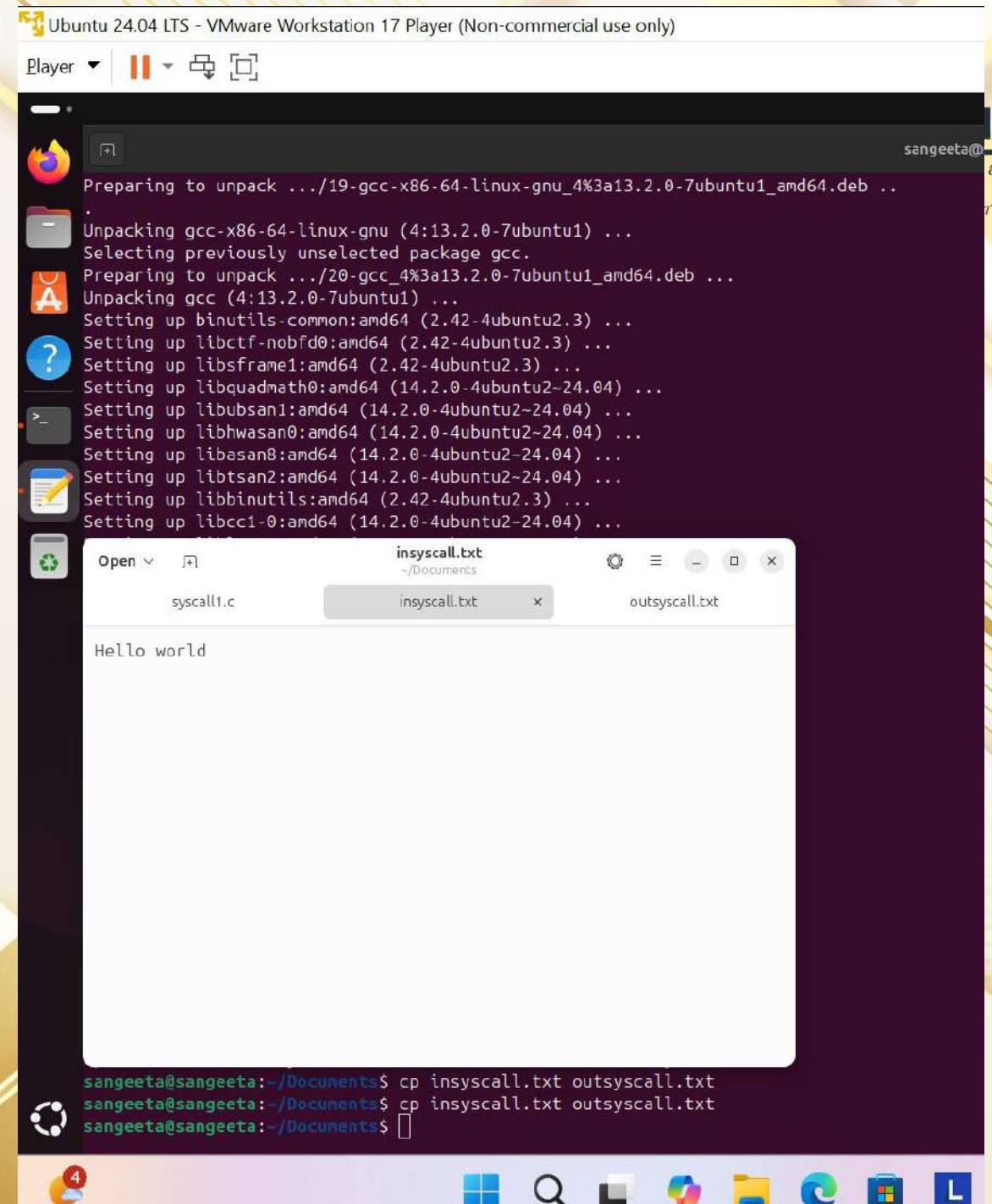

```

100.00      0.000888      10      82      4 total
sangeeta@sangeeta:~/Documents$ strace ls
execve("/usr/bin/ls", ["ls"], 0x7fffd7f51530 /* 48 vars */) = 0
brk(NULL)                               = 0x6409e348e000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7e9855966000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=64643, ...}) = 0
mmap(NULL, 64643, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7e9855956000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=174472, ...}) = 0
mmap(NULL, 181960, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7e9855929000
mmap(0x7e985592f000, 118784, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x6000) = 0x7e985592f000
mmap(0x7e985594c000, 24576, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x23000) = 0x7e985594c000
mmap(0x7e9855952000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x29000) = 0x7e9855952000
mmap(0x7e9855954000, 5832, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7e9855954000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7e9855600000
mmap(0x7e9855628000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7e9855628000
mmap(0x7e98557b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x7e98557b0000
mmap(0x7e98557ff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7e98557ff000
mmap(0x7e9855805000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7e9855805000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=625344, ...}) = 0
mmap(NULL, 627472, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7e985588f000
mmap(0x7e9855891000, 450560, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) = 0x7e9855891000
mmap(0x7e98558ff000, 163840, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x70000) = 0x7e98558ff000
mmap(0x7e9855927000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x97000) = 0x7e9855927000
close(3)                                 = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7e985588c000
arch_prctl(ARCH_SET_FS, 0x7e985588c800) = 0
set_tid_address(0x7e985588cad0)        = 3533
set_robust_list(0x7e985588cae0, 24)    = 0
rseq(0x7e985588d120, 0x20, 0, 0x53053053) = 0
mprotect(0x7e98557ff000, 16384, PROT_READ) = 0
mprotect(0x7e9855927000, 4096, PROT_READ) = 0
mprotect(0x7e9855952000, 4096, PROT_READ) = 0
mprotect(0x6409d800f000, 8192, PROT_READ) = 0

```


Simple system call

1. Create input text file: say (**insyscall.txt**)
2. Type anything say **Hello world**



```
Ubuntu 24.04 LTS - VMware Workstation 17 Player (Non-commercial use only)

Player | [Pause] | [Full Screen] | [Refresh]

sangeeta@
Preparing to unpack .../19-gcc-x86-64-linux-gnu_4%3a13.2.0-7ubuntu1_amd64.deb ...
Unpacking gcc-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Selecting previously unselected package gcc.
Preparing to unpack .../20-gcc_4%3a13.2.0-7ubuntu1_amd64.deb ...
Unpacking gcc (4:13.2.0-7ubuntu1) ...
Setting up binutils-common:amd64 (2.42-4ubuntu2.3) ...
Setting up libctf-nobfd0:amd64 (2.42-4ubuntu2.3) ...
Setting up libsframe1:amd64 (2.42-4ubuntu2.3) ...
Setting up libquadmath0:amd64 (14.2.0-4ubuntu2~24.04) ...
Setting up libubsan1:amd64 (14.2.0-4ubuntu2~24.04) ...
Setting up libhwasan0:amd64 (14.2.0-4ubuntu2~24.04) ...
Setting up libasan8:amd64 (14.2.0-4ubuntu2~24.04) ...
Setting up libtsan2:amd64 (14.2.0-4ubuntu2~24.04) ...
Setting up libbinutils:amd64 (2.42-4ubuntu2.3) ...
Setting up libcc1-0:amd64 (14.2.0-4ubuntu2~24.04) ...

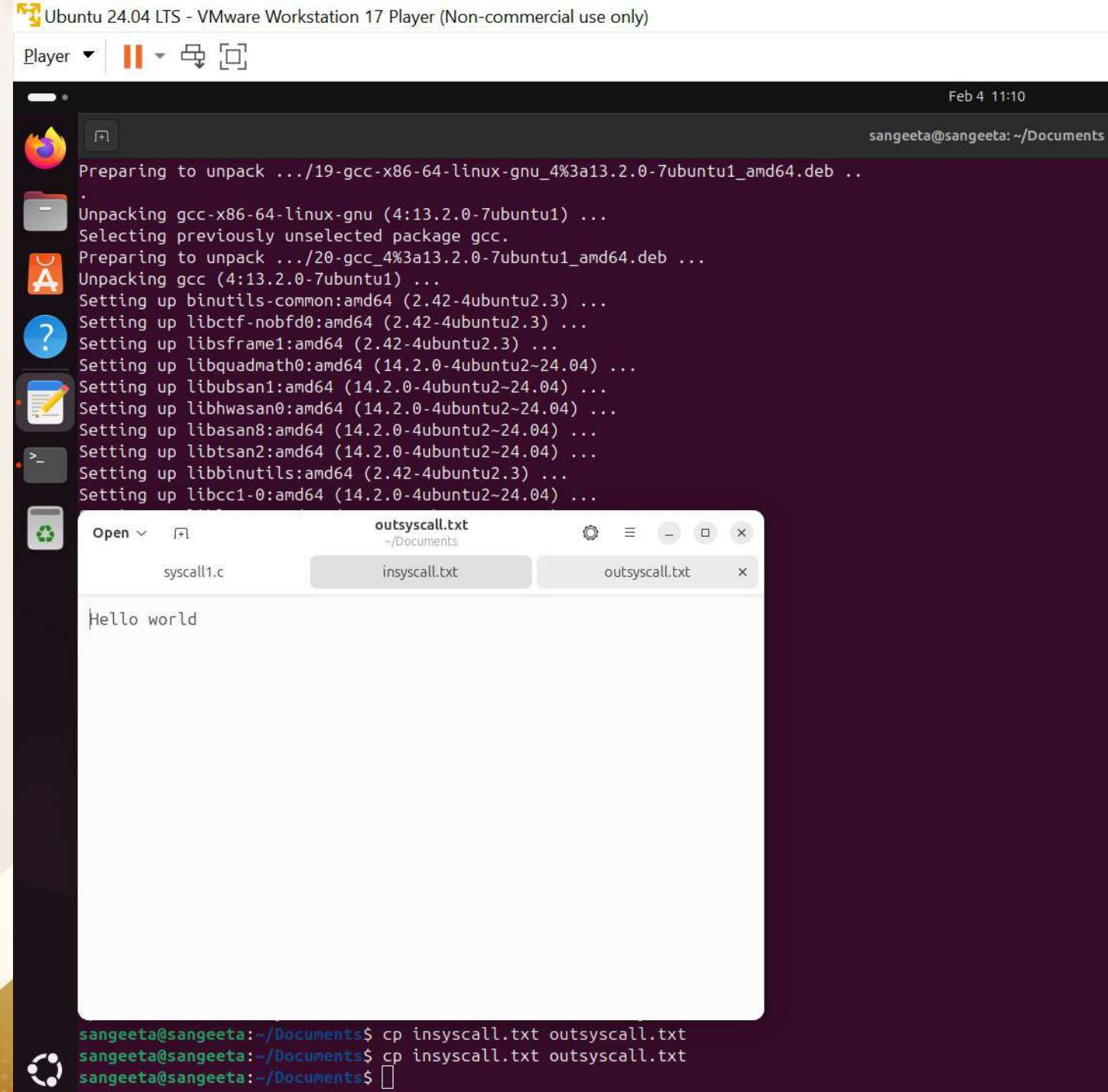
insyscall.txt
~/Documents
syscall1.c  insyscall.txt  outsyscall.txt

Hello world

sangeeta@sangeeta:~/Documents$ cp insyscall.txt outsyscall.txt
sangeeta@sangeeta:~/Documents$ cp insyscall.txt outsyscall.txt
sangeeta@sangeeta:~/Documents$
```


Simple system call

1. Create output text file:
say ([outsyscall.txt](#))
2. Type command: `cp`
[insyscall.txt](#)
[outsyscall.txt](#)
3. The above command
will copy the contents of
the input file
[insyscall.txt](#) to
[outsyscall.txt](#)



The screenshot shows a VMware Workstation 17 Player window titled "Ubuntu 24.04 LTS - VMware Workstation 17 Player (Non-commercial use only)". Inside the player, there is a terminal window with a dark background and a text editor window titled "outsyscall.txt".

The terminal window shows the following output:

```
Preparing to unpack .../19-gcc-x86-64-linux-gnu_4%3a13.2.0-7ubuntu1_amd64.deb ...  
Unpacking gcc-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...  
Selecting previously unselected package gcc.  
Preparing to unpack .../20-gcc_4%3a13.2.0-7ubuntu1_amd64.deb ...  
Unpacking gcc (4:13.2.0-7ubuntu1) ...  
Setting up binutils-common:amd64 (2.42-4ubuntu2.3) ...  
Setting up libctf-nobfd0:amd64 (2.42-4ubuntu2.3) ...  
Setting up libquadmath0:amd64 (14.2.0-4ubuntu2~24.04) ...  
Setting up libubsan1:amd64 (14.2.0-4ubuntu2~24.04) ...  
Setting up libhwasan0:amd64 (14.2.0-4ubuntu2~24.04) ...  
Setting up libasan8:amd64 (14.2.0-4ubuntu2~24.04) ...  
Setting up libtsan2:amd64 (14.2.0-4ubuntu2~24.04) ...  
Setting up libbinutils:amd64 (2.42-4ubuntu2.3) ...  
Setting up libcc1-0:amd64 (14.2.0-4ubuntu2~24.04) ...
```

The text editor window shows the contents of "outsyscall.txt", which is "Hello world".

At the bottom of the terminal window, the following commands are entered:

```
sangeeta@sangeeta:~/Documents$ cp insyscall.txt outsyscall.txt  
sangeeta@sangeeta:~/Documents$ cp insyscall.txt outsyscall.txt  
sangeeta@sangeeta:~/Documents$
```


Types of System Calls

- 1.Process Control** – Create, terminate, load, execute processes. Examples include: `fork()`, `exec()`, `exit()`.
- 2.File Management** – Open, read, write, close files (Examples include: `open()`, `read()`, `write()`, `close()`).
- 3.Device Management** – Request and release devices (Examples include: `ioctl()`, `read()`, `write()`).
- 4.Information Maintenance** – Get/set system information (Examples include: `getpid()`, `alarm()`, `sleep()`).
- 5.Communication** – Send/receive messages between processes (Examples include: `pipe()`, `msgsnd()`, `msgrcv()`).

Why are system call important?

- They enable interaction between user programs and hardware.
- Ensure security and controlled access to system resources.
- Provide essential services like file I/O, memory management, and process control.

Q. Why Applications Are Operating-System Specific?

- Applications compiled on one operating system are not executable on other operating systems.
- Each operating system provides a unique set of system calls. System calls are part of the set of services provided by operating systems for use by applications.

Solution: 1) The application can be written in an **interpreted language** (such as Python or Ruby) that has an interpreter available for multiple operating systems.

2) The application can be written in a language that includes a **virtual machine** containing the running application. Example, java.

3) The application developer can use a **standard language or API** in which the **compiler generates binaries in a machine- and operating-system specific language**.

References

- Silberschatz, Galvin, Gagne, Operating System Concepts, John Wiley and Sons, 10th edition, 2023, ISBN: 935746056X
- Modern Operating Systems by Tanenbaum and Bos , 4th edition, Pearson Publisher, ISBN-13: 978-0-13-359162-0